数据结构竞赛板子（线段树仅含李超树）

本板子默认 C++17。除非单节另有说明，数组下标、点编号和闭区间都从 `1` 开始；涉及权值、计数和区间和时统一使用 `ll`。

代码以赛场直接复制为目标：变量名短、接口集中、避免递归过深；代码块内只写传参和关键边界的短注释，完整用法写在代码块外。

0. 公共约定与选择表

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll INF = (1LL << 62);
```

问题	推荐结构	单次复杂度
动态连通、合并集合	并查集	均摊 `O(alpha(n))`
离线动态连通、需要撤销	可撤销并查集	`O(log n)`，撤销 `O(1)`
维护点之间的差值/奇偶关系	带权并查集	均摊 `O(alpha(n))`
单点加、前缀和/区间和	树状数组	`O(log n)`
区间加、区间和	双树状数组	`O(log n)`
静态幂等区间查询	ST 表	`O(1)`
静态任意结合律区间查询	Disjoint Sparse Table	`O(1)`
无逆元的滑动窗口聚合	SWAG 双栈队列	均摊 `O(1)`
动态中位数、绝对距离和	双 `multiset`	`O(log n)`
动态序列分裂、合并、翻转	隐式 FHQ Treap	期望 `O(log n)`
动态有序多重集	PBDS / FHQ Treap	`O(log n)` / 期望 `O(log n)`
带历史版本的有序多重集	可持久化 FHQ Treap	每次期望 `O(log n)` 新节点
静态区间第 k 小、频次、前驱后继	小波矩阵	`O(log sigma)`
按区间最值递归分治、RMQ 转 LCA	笛卡尔树	建树 `O(n)`
动态区间第 k 小、点修改	有序分块	`O(sqrt(n) log n log sigma)`
动态直线/线段最值	李超树	直线 `O(log n)`，线段 `O(log^2 n)`

问题	推荐结构	单次复杂度
动态树连边、删边、路径信息	LCT	均摊 `O(log n)`
静态树子树颜色统计	DSU on tree	`O(n log n)`
高维最近点、矩形统计	KD Tree	随机数据期望较快
大量离线区间询问	莫队	常见 `O((n+q)sqrt(n))`
大量离线树上路径询问	树上莫队	常见 `O((n+q)sqrt(n))`
三维偏序计数	CDQ + 树状数组	`O(n log^2 n)`

若只需标准容器能完成的功能，优先使用 `vector`、`deque`、`set`、`priority_queue`。复杂结构只有在功能或常数确实需要时再上。

---

1. 坐标离散化

把任意可比较值映射为连续编号，常用于树状数组、计数数组和离线询问。相等值映射到同一编号，编号从 `1` 开始。

- `Disc<T> d(a)`：收集并去重数组 `a`。
- `id(x)`：返回 `x` 的精确编号；调用前应保证 `x` 已加入。
- `lower(x)`：返回第一个大于等于 `x` 的值的编号，结果范围 `1..size()+1`。
- `val(i)`：由编号还原原值。

构造 `O(n log n)`，单次映射 `O(log n)`。

```
template<class T> struct Disc {
    vector<T> v;
    Disc() {}
    Disc(vector<T> a) { init(move(a)); }
    void init(vector<T> a) { // a: 所有可能出现的值
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
        v = move(a);
    }
    int size() const { return (int)v.size(); }
    int lower(const T &x) const { // 首个 >=x 的编号
        return lower_bound(v.begin(), v.end(), x) - v.begin() + 1;
    }
    int id(const T &x) const { return lower(x); } // x 必须存在
    const T &val(int i) const { return v[i - 1]; } // i: 1-based 编号
};
```

只用一次时可直接写：

```
vector<ll> b = a;
sort(b.begin(), b.end());
b.erase(unique(b.begin(), b.end()), b.end());
int id = lower_bound(b.begin(), b.end(), x) - b.begin() + 1; // x 的编号
```

---

2. 并查集

2.1 普通并查集 DSU

维护无向连通块，支持合并、判连通、查询块大小。不支持删边。路径压缩加按大小合并，均摊复杂度 `O(alpha(n))`，实际近似常数。

- `DSU d(n)`：建立 `1..n` 共 `n` 个单点集合。
- `find(x)`：返回代表元。
- `merge(a,b)`：合并，原本已连通则返回 `false`。
- `same(a,b)`、`size(x)`：判连通、查询连通块点数。

```
struct DSU {
    vector<int> p, sz;
    DSU(int n = 0) { init(n); }
    void init(int n) { // n: 元素个数
        p.resize(n + 1);
        iota(p.begin(), p.end(), 0);
        sz.assign(n + 1, 1);
    }
    int find(int x) { // x: 元素编号
        while (x != p[x]) x = p[x] = p[p[x]];
        return x;
    }
    bool merge(int a, int b) { // 合并 a,b
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a, sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
};
```

2.2 可撤销并查集 Rollback DSU

用于分治时间轴、线段树分治等离线动态连通问题。它不能路径压缩，否则一次撤销要恢复大量父亲；按大小合并保证树高 `O(log n)`。

- `time()`：保存当前版本号。
- `merge(a,b)`：合并并记录历史；即使合并失败也不会产生记录。

- `undo(t)`：撤销到版本 `t`。
- `cc`：当前连通块数量。

一次查询/合并 `O(log n)`，一次成功合并的撤销 `O(1)`。

```
struct RDSU {
    struct H { int a, b, sb; };
    vector<int> p, sz;
    vector<H> h;
    int cc;
    RDSU(int n = 0) { init(n); }
    void init(int n) { // n: 元素个数
        p.resize(n + 1);
        iota(p.begin(), p.end(), 0);
        sz.assign(n + 1, 1);
        h.clear(), cc = n;
    }
    int find(int x) { // 不做路径压缩
        while (x != p[x]) x = p[x];
        return x;
    }
    int time() const { return (int)h.size(); }
    bool merge(int a, int b) { // 合并 a,b
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        h.push_back({a, b, sz[b]});
        p[b] = a, sz[a] += sz[b], cc--;
        return true;
    }
    void undo(int t) { // t: time() 保存的版本
        while ((int)h.size() > t) {
            auto [a, b, sb] = h.back(); h.pop_back();
            p[b] = b, sz[a] -= sb, cc++;
        }
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
};
```

典型调用顺序为 `int t=d.time();`，执行若干次 `merge` 并递归处理子问题，返回时调用 `d.undo(t)`。

2.3 带权并查集

维护形如 `value[y] - value[x] = w` 的差分约束。`d[x]` 表示 `value[x] - value[parent[x]]`；路径压缩后表示到根的差。

- `merge(x,y,w)`：加入 `value[y]-value[x]=w`。若合并成功返回 `true`；已连通时只检查约束是否一致。

- `same(x,y)`：是否能推出两点差值。
- `diff(x,y)`：返回 `value[y]-value[x]`，调用前必须已连通。

若只维护奇偶关系，把所有 `ll` 权值改为 `int`，并把加减改成异或即可。均摊复杂度 `O(alpha(n))`。

```
struct WDSU {
    vector<int> p, sz;
    vector<ll> d;
    WDSU(int n = 0) { init(n); }
    void init(int n) { // n: 元素个数
        p.resize(n + 1);
        iota(p.begin(), p.end(), 0);
        sz.assign(n + 1, 1);
        d.assign(n + 1, 0);
    }
    int find(int x) {
        if (x == p[x]) return x;
        int f = p[x];
        p[x] = find(f), d[x] += d[f];
        return p[x];
    }
    bool merge(int x, int y, ll w) { // 约束 val[y]-val[x]=w
        int a = find(x), b = find(y);
        ll dx = d[x], dy = d[y];
        if (a == b) return dy - dx == w;
        if (sz[a] < sz[b]) {
            p[a] = b, d[a] = dy - dx - w, sz[b] += sz[a];
        } else {
            p[b] = a, d[b] = w + dx - dy, sz[a] += sz[b];
        }
        return true;
    }
    bool same(int x, int y) { return find(x) == find(y); }
    ll diff(int x, int y) { // 返回 val[y]-val[x]，须连通
        find(x), find(y);
        return d[y] - d[x];
    }
};
```

注意：`merge` 在已连通且约束冲突时也返回 `false`，与普通 DSU 的“没有发生合并”语义不同；这里的返回值表示“加入约束后仍一致”。

2.4 后继并查集：批量删除/跳过已处理位置

维护“从 `x` 开始第一个未删除位置”。常用于区间染色、离线赋值、快速跳过已经处理过的下标。每个位置只能删除一次。

- 建立 `NextDSU d(n)` 后，有效位置为 `1..n`，`n+1` 是哨兵。
- `next(x)`：第一个不小于 `x` 的未删除位置。
- `erase(x)`：删除位置 `x`。

- 处理整个 `[l,r]`：循环 `for (int i=d.next(l); i<=r; i=d.next(i)) { ...; d.erase(i); }`。

所有删除和跳转合计 `O(n alpha(n))`。

```
struct NextDSU {
    vector<int> p;
    NextDSU(int n = 0) { init(n); }
    void init(int n) { // 有效位置 1..n，n+1 为哨兵
        p.resize(n + 2);
        iota(p.begin(), p.end(), 0);
    }
    int next(int x) { return p[x] == x ? x : p[x] = next(p[x]); }
    void erase(int x) { p[next(x)] = next(x + 1); } // x: 未删除位置
};
```

---

3. 树状数组 Fenwick Tree

3.1 单点加、区间和、前缀第 k 个

`BIT<T>` 维护可加减的信息，最常用为频次和区间和。下标必须从 `1` 开始。

- `add(x,v)`：`a[x] += v`。
- `sum(x)`：返回 `a[1]+...+a[x]`，允许 `x=0`。
- `sum(l,r)`：闭区间和。
- `kth(k)`：在所有频次非负时，返回最小的 `x` 使 `sum(x)>=k`；要求 `1<=k<=sum(n)`。

修改和查询 `O(log n)`，建空树 `O(n)`。从数组建树时，逐点 `add` 是 `O(n log n)`；下方 `build` 为 `O(n)`。

```
template<class T> struct BIT {
    int n;
    vector<T> t;
    BIT(int n = 0) { init(n); }
    void init(int n_) { n = n_, t.assign(n + 1, T()); }
    void build(const vector<T> &a) { // a[1..n]
        init((int)a.size() - 1);
        for (int i = 1; i <= n; i++) {
            t[i] += a[i];
            int j = i + (i & -i);
            if (j <= n) t[j] += t[i];
        }
    }
    void add(int x, T v) { // a[x]+=v
        for (; x <= n; x += x & -x) t[x] += v;
    }
    T sum(int x) const { // [1,x]
        T s = T();
```

```
        for (; x > 0; x -= x & -x) s += t[x];
        return s;
    }
    T sum(int l, int r) const { // [l,r]
        return l > r ? T() : sum(r) - sum(l - 1);
    }
    int kth(T k) const { // 最小 x 使前缀和 >=k
        int x = 0, d = 1;
        T s = T();
        while ((d << 1) <= n) d <<= 1;
        for (; d; d >>= 1) {
            int y = x + d;
            if (y <= n && s + t[y] < k) x = y, s += t[y];
        }
        return x + 1;
    }
};
```

逆序对：先离散化；从左到右处理 `a[i]`，答案增加“已出现元素数减去 `<=a[i]` 的数量”，再将该值频次加一。答案要用 `ll`。

3.2 区间加、区间和

用两个树状数组维护差分，支持闭区间 `[l,r]` 加 `v` 与闭区间求和。

- `add(l,r,v)`：区间每个数加 `v`。
- `sum(x)`：前缀 `[1,x]` 的元素和。
- `sum(l,r)`：闭区间元素和。

每次操作 `O(log n)`，空间 `O(n)`。

```
template<class T> struct RBIT {
    int n;
    BIT<T> a, b;
    RBIT(int n = 0) : n(n), a(n), b(n) {}
    void add1(BIT<T> &t, int x, T v) {
        if (x <= n) t.add(x, v);
    }
    void add(int l, int r, T v) { // [l,r]+=v
        add1(a, l, v), add1(a, r + 1, -v);
        add1(b, l, v * (l - 1)), add1(b, r + 1, -v * r);
    }
    T sum(int x) const { // sum of [1,x]
        return a.sum(x) * x - b.sum(x);
    }
    T sum(int l, int r) const { // sum of [l,r]
        return l > r ? T() : sum(r) - sum(l - 1);
    }
};
```

若有初始数组，可以对每个 `i` 调用 `add(i,i,a[i])`。若只需“区间加、单点查”，一个普通 BIT 维护差分即可：`add(l,v)`, `add(r+1,-v)`，点值是 `sum(x)`。

3.3 二维树状数组：单点加、矩形和

适用于坐标范围较小的二维点权。`add(x,y,v)` 修改一个点，`sum(x1,y1,x2,y2)` 查询闭矩形。操作 `O(log n log m)`，空间 `O(nm)`；若 `nm` 很大，应改用离线扫描线、动态开点或每层离散化。

```
template<class T> struct BIT2 {
    int n, m;
    vector<vector<T>> t;
    BIT2(int n = 0, int m = 0) : n(n), m(m), t(n + 1, vector<T>(m + 1)) {}
    void add(int x, int y, T v) { // point (x,y)+=v
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) t[i][j] += v;
    }
    T pre(int x, int y) const { // [1,x] * [1,y]
        T s = T();
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) s += t[i][j];
        return s;
    }
    T sum(int x1, int y1, int x2, int y2) const { // 闭矩形
        return pre(x2, y2) - pre(x1 - 1, y2)
             - pre(x2, y1 - 1) + pre(x1 - 1, y1 - 1);
    }
};
```

3.4 二维树状数组：矩形加、矩形和

二维差分配合四棵树状数组，支持闭矩形统一加值和闭矩形求和。为降低常数，四棵树都使用连续的一维内存；下标 `(x,y)` 映射为 `x*(m+1)+y`。

- `BIT2R d(n,m)`：坐标范围 `[1,n] * [1,m]`。
- `add(x1,y1,x2,y2,v)`：闭矩形每个点加 `v`。
- `sum(x1,y1,x2,y2)`：闭矩形元素和。

单次操作 `O(log n log m)`，空间约 `32(n+1)(m+1)` 字节。空间是主要限制，例如 `2000*2000` 已约占 `128 MB`。

```
struct BIT2R {
    int n, m, w;
    vector<ll> a, bx, by, bxy;
    BIT2R(int n = 0, int m = 0) : n(n), m(m), w(m + 1) {
        ll z = (ll)(n + 1) * (m + 1);
        a.assign(z, 0), bx.assign(z, 0), by.assign(z, 0), bxy.assign(z, 0);
    }
    void add1(vector<ll> &t, int x, int y, ll v) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) t[(ll)i * w + j] += v;
```

```
    }
    ll ask1(const vector<ll> &t, int x, int y) const {
        ll z = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) z += t[(ll)i * w + j];
        return z;
    }
    void upd(int x, int y, ll v) { // 二维差分点 (x,y)+=v
        add1(a, x, y, v);
        add1(bx, x, y, v * (x - 1));
        add1(by, x, y, v * (y - 1));
        add1(bxy, x, y, v * (x - 1) * (y - 1));
    }
    void add(int x1, int y1, int x2, int y2, ll v) { // 闭矩形 +=v
        upd(x1, y1, v), upd(x2 + 1, y1, -v);
        upd(x1, y2 + 1, -v), upd(x2 + 1, y2 + 1, v);
    }
    ll pre(int x, int y) const { // sum [1,x]*[1,y]
        return (ll)x * y * ask1(a, x, y) - (ll)y * ask1(bx, x, y)
             - (ll)x * ask1(by, x, y) + ask1(bxy, x, y);
    }
    ll sum(int x1, int y1, int x2, int y2) const { // 闭矩形和
        return pre(x2, y2) - pre(x1 - 1, y2)
             - pre(x2, y1 - 1) + pre(x1 - 1, y1 - 1);
    }
};
```

3.5 树状数组维护单调前缀最值

当单点值只会向更优方向变化时，树状数组可以维护前缀最大/最小值，常用于 LIS 型 DP、按坐标扫描的最优转移。

- `BITBest<T,less<T>> d(n,-INF)`：单点 `chmax`、前缀最大值。
- `BITBest<T,greater<T>> d(n,INF)`：单点 `chmin`、前缀最小值。
- `update(x,v)`：用 `v` 改善位置 `x`；不能撤销或向反方向赋值。
- `ask(x)`：查询 `[1,x]` 的最优值。

操作 `O(log n)`，空间 `O(n)`。它不能通过两个前缀答案相减得到任意区间最值。

```
template<class T, class Cmp = less<T>> struct BITBest {
    int n;
    T id;
    Cmp cmp;
    vector<T> t;
    BITBest(int n = 0, T id = T(), Cmp cmp = Cmp())
        : n(n), id(id), cmp(cmp), t(n + 1, id) {}
    void update(int x, T v) { // a[x] 用 v 改善
        for (; x <= n; x += x & -x) if (cmp(t[x], v)) t[x] = v;
    }
    T ask(int x) const { // best of [1,x]
```

```
        T z = id;
        for (; x > 0; x -= x & -x) if (cmp(z, t[x])) z = t[x];
        return z;
    }
};
```

---

4. 静态区间查询

4.1 ST 表：幂等运算

适用于数组不修改、运算满足结合律且幂等的场景，如 `min/max/gcd/bitwise and/or`。区间和不幂等，不能使用重叠的两个块求和。

- `ST<T,F> st(a,op)`：`a` 使用 `0-based`，`op(x,y)` 为合并函数。
- `ask(l,r)`：查询 `0-based` 闭区间 `[l,r]`。

预处理 `O(n log n)`，查询 `O(1)`，空间 `O(n log n)`。

```
template<class T, class F> struct ST {
    int n, lg;
    F op;
    vector<int> pw;
    vector<vector<T>> f;
    ST(const vector<T> &a, F op) : n(a.size()), op(op) {
        pw.resize(n + 1);
        for (int i = 2; i <= n; i++) pw[i] = pw[i >> 1] + 1;
        lg = pw[n] + 1;
        f.assign(lg, vector<T>(n));
        f[0] = a;
        for (int k = 1; k < lg; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                f[k][i] = op(f[k - 1][i], f[k - 1][i + (1 << (k - 1))]);
    }
    T ask(int l, int r) const { // 0-based [l,r]
        int k = pw[r - l + 1];
        return op(f[k][l], f[k][r - (1 << k) + 1]);
    }
};
```

调用示例：

```
auto mn = [](int x, int y) { return min(x, y); };
ST<int, decltype(mn)> st(a, mn); // a 为 0-based
cout << st.ask(l, r) << '\n';    // 0-based 闭区间最小值
```

4.2 Disjoint Sparse Table：任意结合律运算

当运算只有结合律、没有幂等性时使用，例如区间和、区间乘积、矩阵乘法、函数复合。它预处理每层分界点两侧的后缀与前缀，查询时选覆盖两端点的最高不同位。

- 数组与查询均为 `0-based` 闭区间。
- `op` 必须满足结合律；若不满足交换律也可用，代码保持从左到右顺序。

预处理 `O(n log n)`，查询 `O(1)`，空间 `O(n log n)`。

```
template<class T, class F> struct DST {
    int n, lg;
    F op;
    vector<T> a;
    vector<vector<T>> f;
    DST(const vector<T> &a, F op) : n(a.size()), op(op), a(a) {
        lg = n <= 1 ? 0 : 32 - __builtin_clz(n - 1);
        f.assign(lg, vector<T>(n));
        for (int k = 0; k < lg; k++) {
            int len = 1 << (k + 1), h = len >> 1;
            for (int s = 0; s < n; s += len) {
                int m = min(s + h, n), r = min(s + len, n);
                if (m > s) {
                    f[k][m - 1] = a[m - 1];
                    for (int i = m - 2; i >= s; i--) f[k][i] = op(a[i], f[k][i +
1]);
                }
                if (m < r) {
                    f[k][m] = a[m];
                    for (int i = m + 1; i < r; i++) f[k][i] = op(f[k][i - 1], a[i]);
                }
            }
        }
    }
    T ask(int l, int r) const { // 0-based [l,r]
        if (l == r) return a[l];
        int k = 31 - __builtin_clz(l ^ r);
        return op(f[k][l], f[k][r]);
    }
};
```

4.3 64 位块内状压 RMQ

这是参考材料中线性 RMQ 的实用化版本。每 64 个元素用一个单调栈位掩码处理块内最值，跨块时组合左右残块与块级 ST 表。

- 只适用于静态幂等最值查询，默认 `less<T>` 即区间最小值；传 `greater<T>` 可求最大值。
- `RMQ<T,Cmp> q(a)`：`a` 为 `0-based`。
- `ask(l,r)`：查询半开区间 `[l,r)`，要求 `l<r`。

预处理 `O(n+(n/64)log(n/64))`，查询 `O(1)`；实际内存远小于对每个元素建 ST 表。块大小必须不超过 64，因为一个 `uint64_t` 保存块内单调栈。

```
template<class T, class Cmp = less<T>> struct RMQ {
    static const int B = 64;
    using ull = uint64_t;
    int n, nb, lg;
    Cmp cmp;
    vector<T> v, pre, suf;
    vector<ull> mask;
    vector<vector<T>> st;
    T pick(const T &a, const T &b) const { return cmp(b, a) ? b : a; }
    RMQ(const vector<T> &a, Cmp cmp = Cmp()) : n(a.size()), cmp(cmp), v(a) {
        pre = suf = a, mask.resize(n);
        if (!n) return;
        nb = (n + B - 1) / B, lg = 1;
        while ((1 << lg) <= nb) lg++;
        st.assign(lg, vector<T>(nb));
        for (int b = 0; b < nb; b++) {
            int l = b * B, r = min(n, l + B);
            st[0][b] = a[l];
            for (int i = l + 1; i < r; i++) st[0][b] = pick(st[0][b], a[i]);
            for (int i = l + 1; i < r; i++) pre[i] = pick(pre[i - 1], a[i]);
            for (int i = r - 2; i >= l; i--) suf[i] = pick(a[i], suf[i + 1]);
            ull s = 0;
            for (int i = l; i < r; i++) {
                while (s) {
                    int j = 63 - __builtin_clzll(s);
                    if (!cmp(a[i], a[l + j])) break;
                    s ^= 1ULL << j;
                }
                s |= 1ULL << (i - l), mask[i] = s;
            }
        }
        for (int k = 1; k < lg; k++)
            for (int i = 0; i + (1 << k) <= nb; i++)
                st[k][i] = pick(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
    T block(int l, int r) const { // 同一块内 [l,r)
        int base = l / B * B;
        ull s = mask[r - 1] >> (l - base);
        return v[l + __builtin_ctzll(s)];
    }
    T ask(int l, int r) const { // 0-based [l,r)
        int x = l / B, y = (r - 1) / B;
        if (x == y) return block(l, r);
        T z = pick(suf[l], pre[r - 1]);
        int L = x + 1, R = y;
        if (L < R) {
            int k = 31 - __builtin_clz(R - L);
            z = pick(z, pick(st[k][L], st[k][R - (1 << k)]));
```

```
        }
        return z;
    }
};
```

5. 单调栈与单调队列

5.1 单调栈：左右第一个更小元素

`mono_min(a)` 返回每个位置左侧和右侧第一个严格小于它的位置；不存在时分别为 `0`、`n+1`。`a` 使用 `1-based`。

这可直接用于柱状图最大矩形、每个元素作为区间最小值的影响范围、贡献法。相等元素会被弹出，因此两侧都是严格更小；做贡献统计时若需要避免重复，常改成一侧严格、一侧非严格。总复杂度 `O(n)`。

```
template<class T> pair<vector<int>, vector<int>> mono_min(const vector<T> &a) {
    int n = (int)a.size() - 1;
    vector<int> l(n + 1), r(n + 1), st;
    for (int i = 1; i <= n; i++) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        l[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    st.clear();
    for (int i = n; i >= 1; i--) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        r[i] = st.empty() ? n + 1 : st.back();
        st.push_back(i);
    }
    return {l, r};
}
```

5.2 单调队列：滑动窗口最值

返回每个长度为 `k` 的窗口最大值，结果依次对应 `[1,k]`、`[2,k+1]` 等窗口。每个下标只进出队一次，总复杂度 `O(n)`。

```
template<class T> vector<T> slide_max(const vector<T> &a, int k) { // a[1..n], 窗长 k
    int n = (int)a.size() - 1;
    deque<int> q;
    vector<T> ans;
    for (int i = 1; i <= n; i++) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

求最小值时把 `<=` 改为 `>=`。若窗口长度变化但左右端点都只单调右移，也可直接维护同样的下标双端队列。

5.3 动态中位数与绝对距离和

两个 `multiset` 分别保存较小的一半 `a` 和较大的一半 `b`，始终保持 `|a|=|b|` 或 `|a|=|b|+1`。适合滑动窗口中位数、动态最小绝对偏差。

- `add(x)`、`erase(x)`：插入、删除一个存在的 `x`。
- `median()`：返回下中位数；偶数个元素时返回中间两个数中较小者。
- `cost()`：返回 `sum |x-median|`。

插入、删除 `O(log n)`，查询中位数和距离和 `O(1)`。

```
struct Median {
    multiset<ll> a, b;
    ll sa = 0, sb = 0;
    void balance() {
        while (a.size() < b.size()) {
            auto it = b.begin();
            sa += *it, sb -= *it, a.insert(*it), b.erase(it);
        }
        while (a.size() > b.size() + 1) {
            auto it = prev(a.end());
            sa -= *it, sb += *it, b.insert(*it), a.erase(it);
        }
    }
    void add(ll x) { // 插入 x
        if (a.empty() || x <= *a.rbegin()) a.insert(x), sa += x;
        else b.insert(x), sb += x;
        balance();
    }
    void erase(ll x) { // 删除一个存在的 x
        auto it = a.find(x);
        if (it != a.end()) sa -= x, a.erase(it);
        else it = b.find(x), sb -= x, b.erase(it);
        balance();
    }
    ll median() const { return *a.rbegin(); } // 非空时调用
    ll cost() const { // sum |x-median|
        ll m = median();
        return m * (ll)a.size() - sa + sb - m * (ll)b.size();
    }
    int size() const { return a.size() + b.size(); }
};
```

5.4 SWAG：任意结合律滑动窗口聚合

当窗口维护的运算只有结合律、没有逆元时，不能用"前缀答案相减"。SWAG 用两个带聚合值的栈模拟队列，每个元素均摊只搬运一次。

- `AggQueue<T,F> q(op)`：`op(a,b)` 必须满足结合律，不要求交换律或幂等。
- `push(x)`：从队尾加入；`pop()`：删除队首。
- `fold()`：按队首到队尾顺序聚合全部元素，非空时调用。

单次操作均摊 `O(1)` 次 `op`，空间 `O(窗口长度)`。适用于滑动窗口 `gcd/min/max`、矩阵乘积、函数复合等。

```
template<class T, class F> struct AggQueue {
    vector<pair<T, T>> in, out; // {值, 栈聚合}
    F op;
    AggQueue(F op) : op(op) {}
    void push(const T &x) { // 队尾加入 x
        T s = in.empty() ? x : op(in.back().second, x);
        in.push_back({x, s});
    }
    void move() {
        if (!out.empty()) return;
        while (!in.empty()) {
            T x = in.back().first; in.pop_back();
            T s = out.empty() ? x : op(x, out.back().second);
            out.push_back({x, s});
        }
    }
    void pop() { move(), out.pop_back(); } // 删除队首，须非空
    T fold() const { // 聚合整个非空队列
        if (out.empty()) return in.back().second;
        if (in.empty()) return out.back().second;
        return op(out.back().second, in.back().second);
    }
    int size() const { return in.size() + out.size(); }
    bool empty() const { return in.empty() && out.empty(); }
};
```

---

6. 根号分块：区间加、区间和

将数组按约 `sqrt(n)` 大小分块，维护每块总和与整块加法标记。适合操作容易整块维护、但不值得写复杂树结构的题。

- `SqrtDec d(a)`：`a[1..n]`。
- `add(l,r,v)`：闭区间加。
- `sum(l,r)`：闭区间和。

单次操作 `O(sqrt(n))`，空间 `O(n)`。代码对边界块逐点处理，中间完整块只改标记与块和。

```cpp
struct SqrtDec {
    int n, B, nb;
    vector<ll> a, s, tag;
    vector<int> bel, L, R;
    SqrtDec(const vector<ll> &v) { // v[1..n]
        a = v, n = (int)v.size() - 1;
        B = max(1, (int)sqrt(n)), nb = (n + B - 1) / B;
        s.assign(nb + 1, 0), tag.assign(nb + 1, 0);
        bel.resize(n + 1), L.resize(nb + 1), R.resize(nb + 1);
        for (int b = 1; b <= nb; b++) {
            L[b] = (b - 1) * B + 1, R[b] = min(n, b * B);
            for (int i = L[b]; i <= R[b]; i++) bel[i] = b, s[b] += a[i];
        }
    }
    void add(int l, int r, ll v) { // [l,r]+=v
        int x = bel[l], y = bel[r];
        if (x == y) {
            for (int i = l; i <= r; i++) a[i] += v, s[x] += v;
            return;
        }
        for (int i = l; i <= R[x]; i++) a[i] += v, s[x] += v;
        for (int i = L[y]; i <= r; i++) a[i] += v, s[y] += v;
        for (int b = x + 1; b < y; b++)
            tag[b] += v, s[b] += v * (R[b] - L[b] + 1);
    }
    ll sum(int l, int r) const { // sum [l,r]
        int x = bel[l], y = bel[r];
        ll z = 0;
        if (x == y) {
            for (int i = l; i <= r; i++) z += a[i] + tag[x];
            return z;
        }
        for (int i = l; i <= R[x]; i++) z += a[i] + tag[x];
        for (int i = L[y]; i <= r; i++) z += a[i] + tag[y];
        for (int b = x + 1; b < y; b++) z += s[b];
        return z;
    }
};
```

若维护区间最值，整块加仍可只改 `tag`，块内散改后重建该块最值。若要求区间第 k 小，可对每块维护排序数组，散修改后重排，查询时二分答案并统计 `<=x` 的数；单次约 `O(sqrt(n) log n log V)`。

6.1 动态区间第 k 小分块

维护点修改、区间 `<=x` 计数和区间第 k 小。每块保存一份有序值数组；点修改在所属块内删除旧值并插入新值。第 k 小在离线收集的所有可能值 `xs` 上二分。

- `SqrtOrd d(a,xs)`：`a[1..n]`，`xs` 必须包含初始值和所有后续修改值。
- `setv(p,v)`：令 `a[p]=v`。
- `count_le(l,r,x)`：闭区间内 `<=x` 的数量。

- `kth(l,r,k)`：闭区间第 `k` 小，`k` 从 `1` 开始。

点修改 `O(sqrt n)`，计数 `O(sqrt n log n)`，第 k 小再乘 `O(log sigma)`。相比动态树套树，它更短、常数稳定，并且没有值域在线扩张问题。

```
struct SqrtOrd {
    int n, B, nb;
    vector<int> a, bel, xs;
    vector<vector<int>> s;
    SqrtOrd(const vector<int> &a, vector<int> xs) : n((int)a.size() - 1), a(a),
xs(move(xs)) {
        this->xs.insert(this->xs.end(), a.begin() + 1, a.end());
        sort(this->xs.begin(), this->xs.end());
        this->xs.erase(unique(this->xs.begin(), this->xs.end()), this->xs.end());
        B = max(1, (int)sqrt(n)), nb = (n + B - 1) / B;
        bel.resize(n + 1), s.resize(nb);
        for (int i = 1; i <= n; i++) bel[i] = (i - 1) / B,
s[bel[i]].push_back(a[i]);
        for (auto &v : s) sort(v.begin(), v.end());
    }
    void setv(int p, int v) { // a[p]=v；v 必须在 xs 中
        auto &b = s[bel[p]];
        b.erase(lower_bound(b.begin(), b.end(), a[p]));
        b.insert(upper_bound(b.begin(), b.end(), v), v);
        a[p] = v;
    }
    int count_le(int l, int r, int x) const { // count a[l..r]<=x
        int z = 0;
        while (l <= r && (l - 1) % B) z += a[l++] <= x;
        while (l + B - 1 <= r) {
            auto &b = s[bel[l]];
            z += upper_bound(b.begin(), b.end(), x) - b.begin(), l += B;
        }
        while (l <= r) z += a[l++] <= x;
        return z;
    }
    int kth(int l, int r, int k) const { // [l,r] 第 k 小，1-based
        int lo = 0, hi = (int)xs.size() - 1;
        while (lo < hi) {
            int m = (lo + hi) >> 1;
            if (count_le(l, r, xs[m]) >= k) hi = m;
            else lo = m + 1;
        }
        return xs[lo];
    }
};
```

7. 动态有序集合

7.1 PBDS：排名树

GNU C++ 的 `tree` 支持按排名取元素和统计严格小于某值的元素数。它不是标准 C++，只适用于支持 `ext/pb_ds` 的 GCC/Clang GNU 环境。

多重集不能直接使用 `less_equal`，那会破坏比较器的严格弱序。正确做法是存 `(值,唯一编号)`：

- `insert({x,++id})`：插入一个 `x`。
- `order_of_key({x,INT_MIN})`：严格小于 `x` 的数量。
- `find_by_order(k)`：返回第 `k+1` 小的迭代器，`k` 从 `0` 开始。
- 删除一个 `x`：先 `lower_bound({x,INT_MIN})`，确认值相等后删除该迭代器。

操作复杂度 `O(log n)`。

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

using pii = pair<int, int>;
using ost = tree<pii, null_type, less<pii>, rb_tree_tag,
                 tree_order_statistics_node_update>;

ost tr;
int uid = 0;
tr.insert({x, ++uid});                                  // 插入 x
int rk = tr.order_of_key({x, INT_MIN}) + 1;             // x 的排名
int kth = tr.find_by_order(k - 1)->first;               // 第 k 小
auto it = tr.lower_bound({x, INT_MIN});                 // 一个 x
if (it != tr.end() && it->first == x) tr.erase(it);     // 删除一个 x
```

查询前驱：`lower_bound({x,INT_MIN})` 后向前一步；查询严格后继：`upper_bound({x,INT_MAX})`。务必先判断是否越界。

7.2 优化 FHQ Treap：有序多重集

当环境没有 PBDS，或需要完整掌控内存与行为时使用。Treap 以键值满足二叉搜索树性质，以随机优先级满足堆性质，期望树高 `O(log n)`。

传统 FHQ 每次插入要对整棵树分裂后再合并，删除则要分裂两次。这里采用《FHQ-Treap 优化》（游知理，2023）的做法：插入先像普通 BST 一样下行，仅当新节点优先级更高时对当前期望 `O(log n)` 大小的子树分裂；删除直接定位目标后合并左右子树。动态 `split/merge` 次数由期望 `3 log n` 降至约 `log log n`，论文实测通常快约 2 至 3 倍。

- `Treap tr(q)`：预留最多 `q` 个插入节点；删除不回收编号，因此 `q` 应不小于总插入次数。
- `insert(x)`、`erase(x)`：插入、删除一个 `x`；不存在时不删。
- `rank(x)`：返回 `x` 的排名，即严格小于 `x` 的个数加一。
- `kth(k)`：第 `k` 小，要求 `1<=k<=size()`。
- `pre(x)`、`suf(x)`：严格前驱、严格后继；不存在分别返回 `-INF`、`INF`。

所有操作期望 `O(log n)`。

```
struct Treap {
    struct N { int l = 0, r = 0, sz = 1; unsigned p = 0; ll v = 0; };
    vector<N> t;
    int rt = 0, tot = 0;
    unsigned sd = 712367821;
    Treap(int m = 0) : t(m + 1) {}
    unsigned rnd() {
        sd ^= sd << 13, sd ^= sd >> 17, sd ^= sd << 5;
        return sd;
    }
    int size(int u) const { return u ? t[u].sz : 0; }
    void pull(int u) { if (u) t[u].sz = size(t[u].l) + size(t[u].r) + 1; }
    int node(ll v) { // 新建值 v
        ++tot, t[tot] = N();
        t[tot].v = v, t[tot].p = rnd();
        return tot;
    }
    void split_le(int u, ll v, int &a, int &b) { // a:<=v, b:>v
        if (!u) return void(a = b = 0);
        if (t[u].v <= v) a = u, split_le(t[u].r, v, t[u].r, b), pull(a);
        else b = u, split_le(t[u].l, v, a, t[u].l), pull(b);
    }
    int merge(int a, int b) { // max(a)<=min(b)
        if (!a || !b) return a | b;
        if (t[a].p < t[b].p) {
            t[a].r = merge(t[a].r, b), pull(a);
            return a;
        }
        t[b].l = merge(a, t[b].l), pull(b);
        return b;
    }
    void ins(int &u, int x) { // 优化插入节点 x
        if (!u) return void(u = x);
        if (t[x].p < t[u].p) {
            split_le(u, t[x].v, t[x].l, t[x].r), u = x;
        } else if (t[x].v < t[u].v) ins(t[u].l, x);
        else ins(t[u].r, x);
        pull(u);
    }
    bool del(int &u, ll v) { // 优化删除一个 v
        if (!u) return false;
        bool ok;
        if (v < t[u].v) ok = del(t[u].l, v);
        else if (v > t[u].v) ok = del(t[u].r, v);
        else return u = merge(t[u].l, t[u].r), true;
        if (ok) pull(u);
        return ok;
    }
    void insert(ll v) { int x = node(v); ins(rt, x); } // 插入 v
    void erase(ll v) { del(rt, v); } // 删除一个 v；不存在则不变
```

```
    int rank(ll v) { // v 的排名
        int u = rt, z = 1;
        while (u) {
            if (t[u].v < v) z += size(t[u].l) + 1, u = t[u].r;
            else u = t[u].l;
        }
        return z;
    }
    ll kth(int k) { // 第 k 小，1-based
        int u = rt;
        while (u) {
            int s = size(t[u].l);
            if (k == s + 1) return t[u].v;
            if (k <= s) u = t[u].l;
            else k -= s + 1, u = t[u].r;
        }
        return INF;
    }
    ll pre(ll v) { // 严格前驱
        int u = rt; ll z = -INF;
        while (u) {
            if (t[u].v < v) z = t[u].v, u = t[u].r;
            else u = t[u].l;
        }
        return z;
    }
    ll suf(ll v) { // 严格后继
        int u = rt; ll z = INF;
        while (u) {
            if (t[u].v > v) z = t[u].v, u = t[u].l;
            else u = t[u].r;
        }
        return z;
    }
};
```

这里让重复值分别占一个节点，代码最短且排名语义直观。如果操作数非常大且重复值很多，可在节点中增加 `cnt`，以减少内存。

7.3 可合并 FHQ Treap：权值频数集合

每个不同权值只占一个节点，`cnt` 记录该权值出现次数；`mxcnt/mxval` 维护子树中的最大频数及对应权值。`join(x,y)` 不要求两棵树的键值区间相交，遇到相同权值时会自动累加频数，适合树上差分、DSU on tree 等需要启发式合并的场景。

- `MergeTreap tr(q)`：预留最多 `q` 个实际创建的不同权值节点；合并相同权值时被吸收的节点不会回收。
- `insert(rt,v,c)`：向根 `rt` 中加入 `c` 个 `v`，默认 `c=1`。
- `join(x,y)`：合并两棵任意 Treap，返回新根；`x`、`y` 合并后不应再单独使用。

- `best_value(rt)`、`best_count(rt)`：查询根 `rt` 子树中频数最多的权值及其频数；并列时取较小权值。

`insert` 期望 `O(log n)`；合并大小分别为 `n`、`m` 的两棵树时，`join` 的期望复杂度为 `O(m log(n/m+1))`，其中取 `m=min(n,m)`。根节点可由外部数组维护，例如 `root[u] = tr.join(root[u], root[v])`。

```
struct MergeTreap {
    struct N {
        int l = 0, r = 0;
        unsigned p = 0;
        ll v = 0;
        int cnt = 0;
        ll mxval = 0;
        int mxcnt = 0;
    };
    vector<N> t;
    int tot = 0;
    unsigned sd = 712367821;

    MergeTreap(int m = 0) : t(m + 1) {}
    unsigned rnd() {
        sd ^= sd << 13, sd ^= sd >> 17, sd ^= sd << 5;
        return sd;
    }
    void pull(int u) {
        if (!u) return;
        t[u].mxcnt = t[u].cnt;
        t[u].mxval = t[u].v;
        if (t[u].l && t[t[u].l].mxcnt >= t[u].mxcnt)
            t[u].mxcnt = t[t[u].l].mxcnt, t[u].mxval = t[t[u].l].mxval;
        if (t[u].r && t[t[u].r].mxcnt > t[u].mxcnt)
            t[u].mxcnt = t[t[u].r].mxcnt, t[u].mxval = t[t[u].r].mxval;
    }
    int node(ll v, int c) {
        ++tot, t[tot] = N();
        t[tot].v = v, t[tot].p = rnd(), t[tot].cnt = c;
        pull(tot);
        return tot;
    }
    void split3(int u, ll v, int &l, int &r, int &mid) {
        if (!u) return void(l = r = mid = 0);
        if (t[u].v < v) {
            l = u;
            split3(t[u].r, v, t[u].r, r, mid), pull(l);
        } else if (t[u].v > v) {
            r = u;
            split3(t[u].l, v, l, t[u].l, mid), pull(r);
        } else {
            mid = u, l = t[u].l, r = t[u].r;
            t[u].l = t[u].r = 0, pull(u);
```

```
        }
    }
    int merge(int a, int b) { // 要求 max(a)<min(b)
        if (!a || !b) return a | b;
        if (t[a].p < t[b].p) {
            t[a].r = merge(t[a].r, b), pull(a);
            return a;
        }
        t[b].l = merge(a, t[b].l), pull(b);
        return b;
    }
    int join(int x, int y) { // 任意两树合并，相同权值频数累加
        if (!x || !y) return x | y;
        if (t[x].p > t[y].p) swap(x, y);
        int l, r, mid;
        split3(y, t[x].v, l, r, mid);
        t[x].l = join(t[x].l, l);
        t[x].r = join(t[x].r, r);
        if (mid) t[x].cnt += t[mid].cnt;
        pull(x);
        return x;
    }
    void insert(int &u, ll v, int c = 1) {
        int l, r, mid;
        split3(u, v, l, r, mid);
        if (mid) t[mid].cnt += c, pull(mid);
        else mid = node(v, c);
        u = merge(merge(l, mid), r);
    }
    ll best_value(int u) const { return t[u].mxval; } // u 非零
    int best_count(int u) const { return t[u].mxcnt; } // u 非零
};
```

例如树上差分中可写 `tr.insert(root[u], value, 1)`，回溯/合并时写 `root[u] = tr.join(root[u], root[v])`。`join` 会破坏并吸收第二棵树的节点；若需要保留旧树，必须另行复制节点实现持久化。

7.4 可合并优化 FHQ Treap：频数、排名与分裂

这一版融合了上面两种写法：不同权值只建立一个节点，插入沿 BST 下行；只有新节点优先级更高时才分裂当前子树。节点同时维护总频数 `sz` 与子树最大频数 `mxcnt/mxval`，并支持按权值分裂、任意合并、排名和第 `k` 小。

- `MergeFHQ tr(q)`：预留最多 `q` 个不同权值节点；节点编号不会回收，`q` 应不小于整个过程中新建的节点数。
- `insert(rt,v,c)`：向根 `rt` 加入 `c` 个 `v`，默认 `c=1`；重复权值不会新建节点。
- `split(rt,v,a,b)`：将 `rt` 分成 `<v` 与 `>=v` 两棵树。
- `merge(a,b)`：合并键值区间不相交的两棵树，要求 `max(a)<min(b)`。
- `join(a,b)`：合并任意两棵树，相同权值的频数自动累加；合并后原根不可继续单独使用。

- `rank(rt,v)`：返回严格小于 `v` 的元素个数加一；`kth(rt,k)`：返回第 `k` 小的权值。
- `best_value(rt)`、`best_count(rt)`：查询出现次数最多的权值及其频数，并列时取较小权值。

插入、分裂、排名和第 `k` 小均为期望 `O(log n)`；两棵树的不同权值节点数为 `n,m` 时，`join` 期望为 `O(m log(n/m+1))`，其中 `m=min(n,m)`。`sz` 和 `cnt` 使用 `ll`，可处理单个权值频数或集合总大小超过 `int` 的情况。

```
struct MergeFHQ {
    struct N {
        int l = 0, r = 0;
        unsigned p = 0;
        ll v = 0, cnt = 0, sz = 0;
        ll mxval = 0, mxcnt = 0;
    };
    vector<N> t;
    int tot = 0;
    mt19937 rng{20061017};

    MergeFHQ(int m = 0) : t(m + 1) {}
    ll size(int u) const { return u ? t[u].sz : 0; }
    void pull(int u) {
        if (!u) return;
        t[u].sz = size(t[u].l) + size(t[u].r) + t[u].cnt;
        t[u].mxcnt = t[u].cnt, t[u].mxval = t[u].v;
        if (t[u].l && t[t[u].l].mxcnt >= t[u].mxcnt)
            t[u].mxcnt = t[t[u].l].mxcnt, t[u].mxval = t[t[u].l].mxval;
        if (t[u].r && t[t[u].r].mxcnt > t[u].mxcnt)
            t[u].mxcnt = t[t[u].r].mxcnt, t[u].mxval = t[t[u].r].mxval;
    }
    int node(ll v, ll c, unsigned p) {
        ++tot, t[tot] = N();
        t[tot].v = v, t[tot].cnt = c, t[tot].p = p;
        pull(tot);
        return tot;
    }
    void split(int u, ll v, int &a, int &b) { // a:<v, b:>=v
        if (!u) return void(a = b = 0);
        if (t[u].v < v) {
            a = u, split(t[u].r, v, t[u].r, b), pull(a);
        } else {
            b = u, split(t[u].l, v, a, t[u].l), pull(b);
        }
    }
    void split3(int u, ll v, int &a, int &b, int &mid) {
        if (!u) return void(a = b = mid = 0);
        if (t[u].v < v) {
            a = u, split3(t[u].r, v, t[u].r, b, mid), pull(a);
        } else if (t[u].v > v) {
            b = u, split3(t[u].l, v, a, t[u].l, mid), pull(b);
        } else {
            mid = u, a = t[u].l, b = t[u].r;
```

```
            t[u].l = t[u].r = 0, pull(u);
        }
    }
    int merge(int a, int b) { // 要求 max(a)<min(b)
        if (!a || !b) return a | b;
        if (t[a].p > t[b].p) {
            t[a].r = merge(t[a].r, b), pull(a);
            return a;
        }
        t[b].l = merge(a, t[b].l), pull(b);
        return b;
    }
    bool add_existing(int u, ll v, ll c) {
        if (!u) return false;
        if (v == t[u].v) {
            t[u].cnt += c, pull(u);
            return true;
        }
        bool ok = v < t[u].v ? add_existing(t[u].l, v, c)
                             : add_existing(t[u].r, v, c);
        if (ok) pull(u);
        return ok;
    }
    void insert_value(int &u, ll v, ll c, unsigned p) {
        if (!u) return void(u = node(v, c, p));
        if (p > t[u].p) {
            if (add_existing(u, v, c)) return;
            int a, b;
            split(u, v, a, b);
            u = node(v, c, p), t[u].l = a, t[u].r = b, pull(u);
            return;
        }
        if (v < t[u].v) insert_value(t[u].l, v, c, p);
        else if (v > t[u].v) insert_value(t[u].r, v, c, p);
        else t[u].cnt += c;
        pull(u);
    }
    void insert(int &u, ll v, ll c = 1) { insert_value(u, v, c, rng()); }
    int join(int x, int y) { // 任意两树合并，相同权值频数累加
        if (!x || !y) return x | y;
        if (t[x].p < t[y].p) swap(x, y);
        int a, b, mid;
        split3(y, t[x].v, a, b, mid);
        t[x].l = join(t[x].l, a), t[x].r = join(t[x].r, b);
        if (mid) t[x].cnt += t[mid].cnt;
        pull(x);
        return x;
    }
    ll rank(int u, ll v) const {
        ll z = 1;
        while (u) {
            if (t[u].v < v) z += size(t[u].l) + t[u].cnt, u = t[u].r;
```

```
            else u = t[u].l;
        }
        return z;
    }
    ll kth(int u, ll k) const { // k: 1-based，须合法
        while (u) {
            ll s = size(t[u].l);
            if (k <= s) u = t[u].l;
            else if (k <= s + t[u].cnt) return t[u].v;
            else k -= s + t[u].cnt, u = t[u].r;
        }
        return INF;
    }
    ll best_value(int u) const { return t[u].mxval; } // u 非零
    ll best_count(int u) const { return t[u].mxcnt; } // u 非零
};
```

按值切出闭区间 `[x,y]` 的写法为：`split(root[p], x, a, b)`，再对 `b` 调用 `split(b, y + 1, mid, c)`；区间树为 `mid`，剩余部分用 `root[p] = tr.merge(a, c)` 合回。树上启发式合并可直接写 `root[u] = tr.join(root[u], root[v])`，并将 `root[v]` 清零。

7.5 可持久化 FHQ Treap：历史版本有序多重集

每次修改只复制搜索路径上的节点，旧根仍可查询。适合“基于某个历史版本插入/删除，再查询该版本排名或第 k 小”的题。

- `PTreap tr(预计节点数)`：参数只用于 `reserve`，不足时 `vector` 会自动扩容。
- `insert(ver,x)`、`erase(ver,x)`：从版本 `ver` 产生新版本，返回新版本号；删除不存在的值时集合不变。
- `copy(ver)`：直接复制一个版本根。
- `rank(ver,x)`、`kth(ver,k)`、`pre(ver,x)`、`suf(ver,x)`：只读查询。

每次修改期望时间与新增节点数均为 `O(log n)`，查询不分配节点。插入继续使用局部分裂优化；节点不可原地修改，否则会污染旧版本。

```
struct PTreap {
    struct N { int l = 0, r = 0, sz = 1; unsigned p = 0; ll v = 0; };
    vector<N> t;
    vector<int> rt;
    unsigned sd = 712367821;
    PTreap(int m = 0) { t.reserve(m + 1), t.push_back(N()), rt.push_back(0); }
    unsigned rnd() {
        sd ^= sd << 13, sd ^= sd >> 17, sd ^= sd << 5;
        return sd;
    }
    int size(int u) const { return u ? t[u].sz : 0; }
    void pull(int u) { t[u].sz = size(t[u].l) + size(t[u].r) + 1; }
    int node(ll v) {
        t.push_back(N());
        int u = (int)t.size() - 1;
```

```
        t[u].v = v, t[u].p = rnd();
        return u;
    }
    int clone(int u) { t.push_back(t[u]); return (int)t.size() - 1; }
    pair<int, int> split_le(int u, ll v) { // 不修改 u；返回 <=v, >v
        if (!u) return {0, 0};
        int x = clone(u);
        if (t[x].v <= v) {
            auto [a, b] = split_le(t[x].r, v);
            t[x].r = a, pull(x);
            return {x, b};
        }
        auto [a, b] = split_le(t[x].l, v);
        t[x].l = b, pull(x);
        return {a, x};
    }
    int merge(int a, int b) { // 持久化合并，max(a)<=min(b)
        if (!a || !b) return a | b;
        if (t[a].p < t[b].p) {
            int x = clone(a);
            t[x].r = merge(t[x].r, b), pull(x);
            return x;
        }
        int x = clone(b);
        t[x].l = merge(a, t[x].l), pull(x);
        return x;
    }
    int ins(int u, int x) { // 从根 u 持久化插节点 x
        if (!u) return x;
        if (t[x].p < t[u].p) {
            auto [a, b] = split_le(u, t[x].v);
            t[x].l = a, t[x].r = b, pull(x);
            return x;
        }
        int y = clone(u);
        if (t[x].v < t[y].v) t[y].l = ins(t[y].l, x);
        else t[y].r = ins(t[y].r, x);
        pull(y);
        return y;
    }
    int del(int u, ll v) { // 删除一个 v，不修改 u
        if (!u) return 0;
        if (t[u].v == v) return merge(t[u].l, t[u].r);
        int x = clone(u);
        if (v < t[x].v) t[x].l = del(t[x].l, v);
        else t[x].r = del(t[x].r, v);
        pull(x);
        return x;
    }
    int insert(int ver, ll v) { // 从 ver 插入 v
        rt.push_back(ins(rt[ver], node(v)));
        return (int)rt.size() - 1;
```

```
    }
    int erase(int ver, ll v) { // 从 ver 删除一个 v
        rt.push_back(del(rt[ver], v));
        return (int)rt.size() - 1;
    }
    int copy(int ver) { rt.push_back(rt[ver]); return (int)rt.size() - 1; }
    int rank(int ver, ll v) const { // v 的排名
        int u = rt[ver], z = 1;
        while (u) {
            if (t[u].v < v) z += size(t[u].l) + 1, u = t[u].r;
            else u = t[u].l;
        }
        return z;
    }
    ll kth(int ver, int k) const { // 第 k 小，1-based
        int u = rt[ver];
        while (u) {
            int s = size(t[u].l);
            if (k == s + 1) return t[u].v;
            if (k <= s) u = t[u].l;
            else k -= s + 1, u = t[u].r;
        }
        return INF;
    }
    ll pre(int ver, ll v) const { // 严格前驱
        int u = rt[ver]; ll z = -INF;
        while (u) {
            if (t[u].v < v) z = t[u].v, u = t[u].r;
            else u = t[u].l;
        }
        return z;
    }
    ll suf(int ver, ll v) const { // 严格后继
        int u = rt[ver]; ll z = INF;
        while (u) {
            if (t[u].v > v) z = t[u].v, u = t[u].l;
            else u = t[u].r;
        }
        return z;
    }
};
```

8. 优化隐式 FHQ Treap：动态序列

以子树大小代替键值，维护序列顺序。支持按位置分裂/合并、插入、删除、闭区间加、赋值、翻转、区间和与区间最值。

- `ITreap tr(q)`：预留最多 `q` 个曾插入节点。
- `insert(pos,v)`：在前 `pos` 个元素之后插入 `v`，`pos` 可为 `0..size()`。

- `erase(l,r)`、`add(l,r,v)`、`assign(l,r,v)`、`rev(l,r)`：参数均是当前序列的 `1-based` 闭区间。
- `sum(l,r)`、`min(l,r)`、`max(l,r)`：查询区间和、最小值、最大值。
- `build(a)`：依次插入 `a` 中所有元素；传入普通 `0-based vector<ll>`。

单次操作期望 `O(log n)`。单点插入同样采用 PDF 中的优化：按位置静态下行，只在新节点优先级更高时分裂当前子树；区间操作仍使用两次分裂。递归深度期望 `O(log n)`；随机种子固定便于复现，若被构造数据攻击可混入时钟种子。

```
struct ITreap {
    struct N {
        int l = 0, r = 0, sz = 1;
        unsigned p = 0;
        ll v = 0, s = 0, mn = INF, mx = -INF;
        ll add = 0, set = 0;
        bool rv = false, hs = false;
    };
    vector<N> t;
    int rt = 0, tot = 0;
    unsigned sd = 712367821;
    ITreap(int m = 0) : t(m + 1) {}
    unsigned rnd() {
        sd ^= sd << 13, sd ^= sd >> 17, sd ^= sd << 5;
        return sd;
    }
    int size(int u) const { return u ? t[u].sz : 0; }
    ll sum(int u) const { return u ? t[u].s : 0; }
    ll min(int u) const { return u ? t[u].mn : INF; }
    ll max(int u) const { return u ? t[u].mx : -INF; }
    int node(ll v) {
        ++tot, t[tot] = N();
        t[tot].v = t[tot].s = t[tot].mn = t[tot].mx = v, t[tot].p = rnd();
        return tot;
    }
    void pull(int u) {
        t[u].sz = size(t[u].l) + size(t[u].r) + 1;
        t[u].s = sum(t[u].l) + sum(t[u].r) + t[u].v;
        t[u].mn = std::min(t[u].v, std::min(min(t[u].l), min(t[u].r)));
        t[u].mx = std::max(t[u].v, std::max(max(t[u].l), max(t[u].r)));
    }
    void apply_add(int u, ll v) {
        if (!u) return;
        t[u].v += v, t[u].s += v * t[u].sz, t[u].mn += v, t[u].mx += v;
        if (t[u].hs) t[u].set += v;
        else t[u].add += v;
    }
    void apply_set(int u, ll v) {
        if (!u) return;
        t[u].v = t[u].mn = t[u].mx = v, t[u].s = v * t[u].sz;
        t[u].set = v, t[u].add = 0, t[u].hs = true;
```

```
    }
    void apply_rev(int u) {
        if (!u) return;
        swap(t[u].l, t[u].r), t[u].rv ^= 1;
    }
    void push(int u) {
        if (t[u].rv) apply_rev(t[u].l), apply_rev(t[u].r), t[u].rv = false;
        if (t[u].hs) {
            apply_set(t[u].l, t[u].set), apply_set(t[u].r, t[u].set);
            t[u].hs = false;
        }
        if (t[u].add) {
            apply_add(t[u].l, t[u].add), apply_add(t[u].r, t[u].add);
            t[u].add = 0;
        }
    }
    void split(int u, int k, int &a, int &b) { // a: 前 k 个
        if (!u) return void(a = b = 0);
        push(u);
        if (size(t[u].l) >= k) b = u, split(t[u].l, k, a, t[u].l), pull(b);
        else a = u, split(t[u].r, k - size(t[u].l) - 1, t[u].r, b), pull(a);
    }
    int merge(int a, int b) { // 序列 a 接序列 b
        if (!a || !b) return a | b;
        if (t[a].p < t[b].p) {
            push(a), t[a].r = merge(t[a].r, b), pull(a);
            return a;
        }
        push(b), t[b].l = merge(a, t[b].l), pull(b);
        return b;
    }
    void ins(int &u, int k, int x) { // 前 k 个元素后插节点 x
        if (!u) return void(u = x);
        push(u);
        if (t[x].p < t[u].p) {
            split(u, k, t[x].l, t[x].r), u = x;
        } else {
            int s = size(t[u].l);
            if (k <= s) ins(t[u].l, k, x);
            else ins(t[u].r, k - s - 1, x);
        }
        pull(u);
    }
    void build(const vector<ll> &a) { // O(n) 建初始序列
        vector<int> st;
        for (ll v : a) {
            int x = node(v), last = 0;
            while (!st.empty() && t[st.back()].p > t[x].p) {
                pull(st.back()), last = st.back(), st.pop_back();
            }
            t[x].l = last;
            if (!st.empty()) t[st.back()].r = x;
```

```
            st.push_back(x);
        }
        for (int i = (int)st.size() - 1; i >= 0; i--) pull(st[i]);
        rt = st.empty() ? 0 : st.front();
    }
    void insert(int pos, ll v) { // 前 pos 个元素后插入 v
        int x = node(v);
        ins(rt, pos, x);
    }
    void erase(int l, int r) { // 删除 [l,r]
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        rt = merge(a, c);
    }
    void add(int l, int r, ll v) { // [l,r]+=v
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        apply_add(b, v), rt = merge(merge(a, b), c);
    }
    void assign(int l, int r, ll v) { // a[l..r]=v
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        apply_set(b, v), rt = merge(merge(a, b), c);
    }
    void rev(int l, int r) { // reverse [l,r]
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        apply_rev(b), rt = merge(merge(a, b), c);
    }
    ll sum(int l, int r) { // sum [l,r]
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        ll z = sum(b);
        rt = merge(merge(a, b), c);
        return z;
    }
    ll min(int l, int r) { // min of [l,r]
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        ll z = min(b);
        rt = merge(merge(a, b), c);
        return z;
    }
    ll max(int l, int r) { // max of [l,r]
        int a, b, c;
        split(rt, r, b, c), split(b, l - 1, a, b);
        ll z = max(b);
        rt = merge(merge(a, b), c);
        return z;
    }
};
```

删除的节点没有回收，但其编号也不会再使用，因此预留容量按“总插入次数”而非“最大同时存在数”计算。

---

9. 可并堆：左偏树

多个优先队列需要频繁合并时，普通 `priority_queue` 无法高效完成；左偏树可在 `O(log n)` 内合并两个堆、插入元素或弹出堆顶。

- `LeftHeap h(q)`：最多新建 `q` 个节点。
- `node(v)`：新建键值为 `v` 的单点堆，返回根编号。
- `merge(x,y)`：合并两个小根堆并返回新根；原根不应再单独使用。
- `top(rt)`：读取最小值；`pop(rt)`：删除最小值并返回新根。

此短版不支持删除任意节点。若题目要求维护堆所属集合，可在外层配普通并查集记录根。

```
struct LeftHeap {
    struct N { int l = 0, r = 0, d = 0; ll v = 0; };
    vector<N> t;
    int tot = 0;
    LeftHeap(int m = 0) : t(m + 1) {}
    int node(ll v) { // 新建单点堆
        ++tot, t[tot] = N(), t[tot].v = v, t[tot].d = 1;
        return tot;
    }
    int merge(int x, int y) { // 合并两个小根堆
        if (!x || !y) return x | y;
        if (t[x].v > t[y].v) swap(x, y);
        t[x].r = merge(t[x].r, y);
        if (t[t[x].l].d < t[t[x].r].d) swap(t[x].l, t[x].r);
        t[x].d = t[t[x].r].d + 1;
        return x;
    }
    ll top(int rt) const { return t[rt].v; } // rt 非零
    int pop(int rt) { return merge(t[rt].l, t[rt].r); } // 删除堆顶
};
```

---

10. Trie

10.1 小写字符串 Trie

维护小写字符串多重集，支持插入、删除、查询完整单词出现次数和前缀出现次数。`cnt` 是经过节点的字符串数，`end` 是在节点结束的字符串数。

单次复杂度 `O(|s|)`，空间与所有插入字符串的总长度同阶。删除前必须保证字符串存在。

```
struct Trie {
    struct N { int ch[26] = {}, cnt = 0, ed = 0; };
```

```
    vector<N> t;
    Trie(int m = 0) { t.reserve(m + 1), t.push_back(N()); }
    void add(const string &s, int v = 1) { // v=1 插入，v=-1 删除
        int u = 0;
        t[u].cnt += v;
        for (char c : s) {
            int x = c - 'a';
            if (!t[u].ch[x]) t[u].ch[x] = t.size(), t.push_back(N());
            u = t[u].ch[x], t[u].cnt += v;
        }
        t[u].ed += v;
    }
    int count(const string &s) const { // 完整串出现次数
        int u = 0;
        for (char c : s) {
            u = t[u].ch[c - 'a'];
            if (!u) return 0;
        }
        return t[u].ed;
    }
    int prefix(const string &s) const { // 以 s 为前缀的串数
        int u = 0;
        for (char c : s) {
            u = t[u].ch[c - 'a'];
            if (!u) return 0;
        }
        return t[u].cnt;
    }
};
```

字符集不止小写字母时，可把 `ch[26]` 改为定长大数组；字符集很大且稀疏时用 `unordered_map<char,int>`，但常数与内存通常更大。

10.2 01 Trie：最大异或、最小异或、异或计数

维护非负整数多重集。默认处理位 `30..0`，即值范围 `[0,2^31)`；若值达到 `1e18`，把 `B` 改为 `62`，键和参数改为 `ll`。

- `add(x,1)` 插入，`add(x,-1)` 删除；删除前必须存在。
- `max_xor(x)`、`min_xor(x)`：返回与集合中某数异或后的最大/最小结果，而不是那个数本身。
- `count_less(x,k)`：集合中满足 `(x xor y) < k` 的 `y` 的数量。

每次操作 `O(B)`。

```
struct XTrie {
    static const int B = 30;
    struct N { int ch[2] = {}, cnt = 0; };
    vector<N> t;
    XTrie(int m = 0) { t.reserve((ll)(m + 1) * (B + 1)), t.push_back(N()); }
    void add(int x, int v = 1) { // v=1 插入，v=-1 删除
        int u = 0; t[u].cnt += v;
```

```
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1;
            if (!t[u].ch[b]) t[u].ch[b] = t.size(), t.push_back(N());
            u = t[u].ch[b], t[u].cnt += v;
        }
    }
    int max_xor(int x) const { // max(x xor y)，集合非空
        int u = 0, z = 0;
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1, v = t[u].ch[b ^ 1];
            if (v && t[v].cnt > 0) z |= 1 << i, u = v;
            else u = t[u].ch[b];
        }
        return z;
    }
    int min_xor(int x) const { // min(x xor y)，集合非空
        int u = 0, z = 0;
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1, v = t[u].ch[b];
            if (v && t[v].cnt > 0) u = v;
            else z |= 1 << i, u = t[u].ch[b ^ 1];
        }
        return z;
    }
    int count_less(int x, int k) const { // count y: (x xor y)<k
        int u = 0, z = 0;
        for (int i = B; i >= 0; i--) {
            int a = x >> i & 1, b = k >> i & 1;
            if (b) {
                int v = t[u].ch[a];
                if (v) z += t[v].cnt;
                u = t[u].ch[a ^ 1];
            } else u = t[u].ch[a];
            if (!u) break;
        }
        return z;
    }
};
```

`count_less` 若 `k<=0` 答案为 `0`；若 `k>=2^(B+1)` 答案为集合大小，这两种超出当前位宽的情况应在调用前特判。

10.3 持久化 01 Trie：静态区间最大异或

为每个前缀保存一个 Trie 根，版本 `rt[i]` 包含 `a[1..i]`。两个版本节点计数相减，就能限制候选元素来自下标区间 `[l,r]`。

- `PXTrie tr(n)`：预留 `n` 次插入所需空间。
- 依次调用 `append(a[i])` 建版本。
- `max_xor(l,r,x)`：返回 `max(x xor a[i])`，其中 `l<=i<=r`。

建树空间 `O(nB)`，构建和查询都是 `O(B)`。默认只处理 `[0,2^31)` 的非负 `int`。

```
struct PXTrie {
    static const int B = 30;
    struct N { int ch[2] = {}, cnt = 0; };
    vector<N> t;
    vector<int> rt;
    PXTrie(int n = 0) {
        t.reserve((ll)(n + 1) * (B + 2));
        t.push_back(N()), rt.push_back(0);
    }
    int clone(int u) { t.push_back(t[u]); return (int)t.size() - 1; }
    void append(int x) { // 新版本加入 x
        int nr = clone(rt.back()), u = nr, old = rt.back();
        t[u].cnt++;
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1, ov = t[old].ch[b];
            int v = clone(ov);
            t[u].ch[b] = v, u = v, old = ov, t[u].cnt++;
        }
        rt.push_back(nr);
    }
    int max_xor(int l, int r, int x) const { // max(x xor a[l..r])
        int u = rt[r], v = rt[l - 1], z = 0;
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1, c = b ^ 1;
            int uc = t[u].ch[c], vc = t[v].ch[c];
            if (t[uc].cnt - t[vc].cnt > 0) z |= 1 << i, u = uc, v = vc;
            else u = t[u].ch[b], v = t[v].ch[b];
        }
        return z;
    }
};
```

11. 异或线性基

11.1 普通线性基

维护一组非负 `ll` 的所有子集异或结果，解决最大/最小异或、表示性判断、不同异或值数量、第 k 小异或值以及两组基合并。

- `insert(x)`：插入，若 `x` 与已有向量线性相关则返回 `false`。
- `can(x)`：`x` 是否能由子集异或得到。
- `max_xor(x)`：在初值 `x` 上异或某个可表示值后的最大结果。
- `min_pos()`：最小正的可表示值；空基返回 `0`。
- `kth(k)`：第 `k` 小的不同子集异或值，`k` 从 `0` 开始且包括 `0`；要求秩小于 63 且 `k<2^rank`。
- `merge(b)`：把线性基 `b` 合入当前基。

插入/查询 `O(B)`，`kth` 的标准化 `O(B^2)`；这里 `B=62`，表示位 `62..0`，要求数值不使用符号位。

```
struct LB {
    static const int B = 62;
    ll d[B + 1] = {};
    int rk = 0;
    bool insert(ll x) { // 插入 x
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!d[i]) return d[i] = x, rk++, true;
            x ^= d[i];
        }
        return false;
    }
    bool can(ll x) const { // x 是否可表示
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!d[i]) return false;
            x ^= d[i];
        }
        return true;
    }
    ll max_xor(ll x = 0) const { // max(x xor subset)
        for (int i = B; i >= 0; i--) x = max(x, x ^ d[i]);
        return x;
    }
    ll min_pos() const { // 最小正异或值
        for (int i = 0; i <= B; i++) if (d[i]) return d[i];
        return 0;
    }
    ll kth(ll k) const { // 0-based，包含 0
        ll b[B + 1];
        copy(d, d + B + 1, b);
        for (int i = 0; i <= B; i++) if (b[i])
            for (int j = i - 1; j >= 0; j--)
                if (b[i] >> j & 1) b[i] ^= b[j];
        vector<ll> p;
        for (int i = 0; i <= B; i++) if (b[i]) p.push_back(b[i]);
        ll z = 0;
        for (int i = 0; i < (int)p.size(); i++) if (k >> i & 1) z ^= p[i];
        return z;
    }
    void merge(const LB &o) { // 合并 o
        for (int i = B; i >= 0; i--) if (o.d[i]) insert(o.d[i]);
    }
};
```

若问“非空子集异或的第 k 小”，需要记录是否出现过线性相关插入：若相关，则非空子集也能得到 `0`；若所有插入向量独立，则应跳过 `kth(0)=0`。

11.2 带来源还原的线性基

在每个基向量旁维护一个 `bitset`，其中第 `id` 位表示该基向量由原始元素 `id` 异或得到。高斯消元时数值和来源集合始终同步异或，因此最后可以还原方案。

- `TraceLB<MAXN> lb`：支持的来源编号范围为 `[0,MAXN)`；`MAXN` 必须是编译期常量。
- `insert(x,id)`：插入值 `x`，来源编号 `id` 必须唯一；独立时返回 `true`，相关时返回 `false`。
- `represent(x)`：若 `x` 可表示，返回一个来源集合；不可表示返回 `nullopt`。
- `max_xor(x)`：返回 `{最大值,来源集合}`；来源集合中元素的异或值是“与初值 `x` 异或的增量”。默认 `x=0` 时就是最大异或子集。
- `min_pos()`：返回 `{最小正异或值,来源集合}`；空基返回数值 `0` 和空集合。
- `kth(k)`：返回第 `k` 小不同异或值及其来源，`k` 从 `0` 开始且包含 `0`。
- `zero()`：若存在非空子集异或为 `0`，返回其来源；所有插入向量都独立时返回 `nullopt`。
- `ids(s,n)`：把来源位集 `s` 转成小于 `n` 的编号列表。

数值消元仍只有 63 位，但每次来源异或要处理 `MAXN/64` 个机器字，因此插入和查询复杂度约为 `O(63*MAXN/64)`。来源位集空间约为 `64*MAXN/8` 字节；`MAXN=200000` 时约 `1.6 MB`。位集存放在 `vector` 的堆内存中，避免大对象压栈。

```
template<int N> struct TraceLB {
    static const int B = 62;
    ll d[B + 1] = {};
    vector<bitset<N>> from; // [0..B] 基向量；B+1 保存零关系
    int rk = 0;
    bool has0 = false;
    TraceLB() : from(B + 2) {}
    bool insert(ll x, int id) { // 插入 x，唯一来源 id
        assert(0 <= id && id < N);
        bitset<N> s;
        s.set(id);
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!d[i]) {
                d[i] = x, from[i] = s, rk++;
                return true;
            }
            x ^= d[i], s ^= from[i];
        }
        if (!has0) from[B + 1] = s, has0 = true;
        return false;
    }
    optional<bitset<N>> represent(ll x) const { // 还原 xor=x 的子集
        bitset<N> s;
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!d[i]) return nullopt;
            x ^= d[i], s ^= from[i];
        }
        return s;
    }
```

```
    pair<ll, bitset<N>> max_xor(ll x = 0) const { // max(x xor subset)
        bitset<N> s;
        for (int i = B; i >= 0; i--) if ((x ^ d[i]) > x)
            x ^= d[i], s ^= from[i];
        return {x, s};
    }
    pair<ll, bitset<N>> min_pos() const { // 最小正异或值及来源
        for (int i = 0; i <= B; i++) if (d[i]) return {d[i], from[i]};
        return {0, {}};
    }
    pair<ll, bitset<N>> kth(ll k) const { // 第 k 小，0-based，包含 0
        vector<ll> b(d, d + B + 1);
        vector<bitset<N>> s(from.begin(), from.begin() + B + 1);
        for (int i = 0; i <= B; i++) if (b[i])
            for (int j = i - 1; j >= 0; j--) if (b[i] >> j & 1)
                b[i] ^= b[j], s[i] ^= s[j];
        vector<int> p;
        for (int i = 0; i <= B; i++) if (b[i]) p.push_back(i);
        ll x = 0;
        bitset<N> src;
        for (int i = 0; i < (int)p.size(); i++) if (k >> i & 1)
            x ^= b[p[i]], src ^= s[p[i]];
        return {x, src};
    }
    optional<bitset<N>> zero() const { // 非空 xor=0 子集
        if (!has0) return nullopt;
        return from[B + 1];
    }
    static vector<int> ids(const bitset<N> &s, int n = N) { // 位集转来源编号
        vector<int> a;
        for (int i = 0; i < n; i++) if (s[i]) a.push_back(i);
        return a;
    }
};
```

典型用法是按输入下标执行 `lb.insert(a[i],i)`。取得 `auto [val,src]=lb.max_xor()` 后，遍历 `TraceLB<MAXN>::ids(src,n)` 就能得到被选中的原始下标；这些下标对应元素的异或值必定等于 `val`。

`represent(0)` 按线性空间定义会返回空集；若要求非空方案，应调用 `zero()`。`max_xor(x)` 返回的来源只负责子集部分，不会把外部初值 `x` 视为某个来源元素。

---

12. 小波矩阵：静态区间顺序统计

小波矩阵适合数组不修改，但需要大量区间第 k 小、频次、前驱后继和值域求和查询。下方版本先离散化值，因此支持负数和任意 `int`，查询复杂度 `O(log sigma)`，空间 `O(n log sigma)`。

数组和区间使用 `0-based` 半开区间 `[l,r)`：

- `WM wm(a)`：构建静态数组。
- `kth(l,r,k)`：区间第 `k+1` 小，`k` 从 `0` 开始。

- `less(l,r,x)`：区间内严格小于 `x` 的数量。
- `freq(l,r,x)`：`x` 的出现次数。
- `range_freq(l,r,lo,hi)`：值域 `[lo,hi)` 内的数量。
- `sum_less(l,r,x)`：区间内所有 `<x` 的元素和。
- `sum_range(l,r,lo,hi)`：区间内值属于 `[lo,hi)` 的元素和。
- `sum_k(l,r,k)`：区间最小的 `k` 个元素之和，`k` 可为 `0..r-l`。
- `pre(l,r,x)`：严格小于 `x` 的最大值，不存在返回 `-INF`。
- `suf(l,r,x)`：大于等于 `x` 的最小值，不存在返回 `INF`。

```
struct WM {
    int n, lg;
    vector<int> xs, z;
    vector<ll> ps;
    vector<vector<int>> p;
    vector<vector<ll>> sm0;
    WM(const vector<int> &a) : n(a.size()) {
        xs = a;
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        int m = max(1, (int)xs.size());
        lg = 1;
        while ((1 << lg) < m) lg++;
        z.resize(lg), p.assign(lg, vector<int>(n + 1));
        ps.assign(n + 1, 0), sm0.assign(lg, vector<ll>(n + 1));
        for (int i = 0; i < n; i++) ps[i + 1] = ps[i] + a[i];
        vector<int> cur(n), nxt(n);
        for (int i = 0; i < n; i++)
            cur[i] = lower_bound(xs.begin(), xs.end(), a[i]) - xs.begin();
        for (int k = lg - 1, lv = 0; k >= 0; k--, lv++) {
            for (int i = 0; i < n; i++) {
                int b = cur[i] >> k & 1;
                p[lv][i + 1] = p[lv][i] + b;
                sm0[lv][i + 1] = sm0[lv][i] + (b ? 0 : xs[cur[i]]);
            }
            z[lv] = n - p[lv][n];
            int a0 = 0, a1 = z[lv];
            for (int x : cur) {
                if (x >> k & 1) nxt[a1++] = x;
                else nxt[a0++] = x;
            }
            cur.swap(nxt);
        }
    }
    int kth_id(int l, int r, int k) const { // [l,r), 0-based k
        int id = 0;
        for (int lv = 0; lv < lg; lv++) {
            int one = p[lv][r] - p[lv][l], zero = r - l - one;
            if (k < zero) l -= p[lv][l], r -= p[lv][r];
```

```
            else {
                id |= 1 << (lg - 1 - lv), k -= zero;
                l = z[lv] + p[lv][l], r = z[lv] + p[lv][r];
            }
        }
        return id;
    }
    int kth(int l, int r, int k) const { // 第 k+1 小
        return xs[kth_id(l, r, k)];
    }
    int less_id(int l, int r, int id) const { // 编号 <id 的数量
        if (id <= 0) return 0;
        if (id >= (int)xs.size()) return r - l;
        int ans = 0;
        for (int lv = 0; lv < lg; lv++) {
            int b = id >> (lg - 1 - lv) & 1;
            int ol = p[lv][l], one = p[lv][r] - ol;
            if (b) {
                ans += r - l - one;
                l = z[lv] + ol, r = z[lv] + p[lv][r];
            } else l -= ol, r -= p[lv][r];
        }
        return ans;
    }
    int less(int l, int r, int x) const { // count value <x
        int id = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        return less_id(l, r, id);
    }
    int freq(int l, int r, int x) const { // count value ==x
        int id = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        if (id == (int)xs.size() || xs[id] != x) return 0;
        return less_id(l, r, id + 1) - less_id(l, r, id);
    }
    int range_freq(int l, int r, int lo, int hi) const { // lo<=value<hi
        return less(l, r, hi) - less(l, r, lo);
    }
    ll sum_less_id(int l, int r, int id) const { // 编号 <id 的元素和
        if (id <= 0) return 0;
        if (id >= (int)xs.size()) return ps[r] - ps[l];
        ll ans = 0;
        for (int lv = 0; lv < lg; lv++) {
            int b = id >> (lg - 1 - lv) & 1;
            int ol = p[lv][l], one = p[lv][r] - ol;
            if (b) {
                ans += sm0[lv][r] - sm0[lv][l];
                l = z[lv] + ol, r = z[lv] + p[lv][r];
            } else l -= ol, r -= p[lv][r];
        }
        return ans;
    }
    ll sum_less(int l, int r, int x) const { // sum of value <x
        int id = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
```

```
        return sum_less_id(l, r, id);
    }
    ll sum_range(int l, int r, int lo, int hi) const { // sum, lo<=value<hi
        return sum_less(l, r, hi) - sum_less(l, r, lo);
    }
    ll sum_k(int l, int r, int k) const { // 最小 k 个元素和
        ll ans = 0;
        int id = 0;
        for (int lv = 0; lv < lg; lv++) {
            int ol = p[lv][l], one = p[lv][r] - ol, zero = r - l - one;
            if (k <= zero) l -= ol, r -= p[lv][r];
            else {
                ans += sm0[lv][r] - sm0[lv][l], k -= zero;
                id |= 1 << (lg - 1 - lv);
                l = z[lv] + ol, r = z[lv] + p[lv][r];
            }
        }
        if (k) ans += (ll)k * xs[id];
        return ans;
    }
    ll pre(int l, int r, int x) const { // max value <x
        int k = less(l, r, x);
        return k ? kth(l, r, k - 1) : -INF;
    }
    ll suf(int l, int r, int x) const { // min value >=x
        int k = less(l, r, x);
        return k < r - l ? kth(l, r, k) : INF;
    }
};
```

13. 笛卡尔树

笛卡尔树同时满足：中序遍历是原数组顺序，堆序由数组值决定。小根树中父节点值不大于子节点值，大根树反之。它把“按区间最小值或最大值递归划分”的结构显式建成一棵树，常用于 RMQ 转 LCA、区间分治和子数组最值贡献。

`Cart(a)` 的参数 `a` 是下标从 `1` 开始的原数组。构造后 `rt` 是根，`l[u]`、`r[u]`、`fa[u]` 分别是节点 `u` 的左右儿子和父亲，`0` 表示不存在。

代码默认建小根树，且相等值不弹栈，所以更靠左的相等元素在上方。要让更靠右的相等元素在上方，将 `>` 改为 `>=`；要建大根树，将 `>` 改为 `<`，相等值规则同理。构建时间和空间都是 $O(n)$。

如果只要左右儿子，可抄第二段极简版，省掉结构体和父指针。

```
template<class T> struct Cart {
    int n, rt = 0;
    vector<int> l, r, fa;
    Cart(const vector<T> &a) { // a[1..n]，建小根笛卡尔树
        n = (int)a.size() - 1;
        l.assign(n + 1, 0), r.assign(n + 1, 0), fa.assign(n + 1, 0);
        vector<int> st;
```

```
        for (int i = 1; i <= n; i++) {
            int x = 0;
            while (!st.empty() && a[st.back()] > a[i]) x = st.back(), st.pop_back();
            if (!st.empty()) r[st.back()] = i, fa[i] = st.back();
            if (x) l[i] = x, fa[x] = i;
            st.push_back(i);
        }
        rt = st.empty() ? 0 : st.front();
    }
};
```

```
// a[1..n]：原数组；返回根节点，l/r 为左右儿子
int build_cart(const vector<int> &a, vector<int> &l, vector<int> &r) {
    int n = (int)a.size() - 1;
    l.assign(n + 1, 0), r.assign(n + 1, 0);
    vector<int> st;
    for (int i = 1; i <= n; i++) {
        int x = 0;
        while (!st.empty() && a[st.back()] > a[i]) x = st.back(), st.pop_back();
        if (!st.empty()) r[st.back()] = i;
        if (x) l[i] = x;
        st.push_back(i);
    }
    return st.empty() ? 0 : st.front();
}
```

单调栈始终维护当前笛卡尔树的右链。插入 `i` 时，最后一个被弹出的节点成为 `i` 的左儿子，弹栈后的栈顶以 `i` 为右儿子。每个节点至多进栈、出栈一次。

统计子数组最小值贡献时，相等元素必须只归一侧：常见做法是一边找严格小于，另一边找小于等于。它与这里选择 `>` 还是 `>=` 是同一件事。

---

14. 静态树上结构

14.1 树链剖分 + 树状数组

将树上路径拆成 `O(log n)` 个 DFS 序连续区间。下方版本用双树状数组支持：路径加、路径和、子树加、子树和。运算必须可交换，因为不同链段没有按路径方向拼接。

- `HLD h(n)`，调用 `add_edge(u,v)` 建无根树，再 `build(root)`。
- `init_val(w)`：写入初始点权 `w[1..n]`，只调用一次。
- `path_add(u,v,x)`、`path_sum(u,v)`：包含两个端点。
- `sub_add(u,x)`、`sub_sum(u)`：以 `build(root)` 的根方向定义子树。
- `lca(u,v)`、`jump(u,k)`、`kth_path(u,v,k)`：LCA、向上跳 `k` 条边、路径上从 `u` 起第 `k` 个点；`k` 从 `0` 开始。
- `root_sub_add(u,rt,x)`、`root_sub_sum(u,rt)`：把根临时看作 `rt` 时，处理 `u` 的子树。

预处理 `O(n)`，路径操作 `O(log^2 n)`，子树操作 `O(log n)`，空间 `O(n)`。两次 DFS 使用递归，链形树可能栈溢出；严格卡栈时应改成显式栈或扩大栈。

```
struct HLD {
    int n, tim = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, sz, son, top, dfn, rid;
    RBIT<ll> bit;
    HLD(int n = 0) : n(n), g(n + 1), fa(n + 1), dep(n + 1),
        sz(n + 1), son(n + 1), top(n + 1), dfn(n + 1), rid(n + 1), bit(0) {}
    void add_edge(int u, int v) { // 无向边 u-v
        g[u].push_back(v), g[v].push_back(u);
    }
    void dfs1(int u, int f) {
        fa[u] = f, dep[u] = dep[f] + 1, sz[u] = 1;
        for (int v : g[u]) if (v != f) {
            dfs1(v, u), sz[u] += sz[v];
            if (!son[u] || sz[v] > sz[son[u]]) son[u] = v;
        }
    }
    void dfs2(int u, int tp) {
        top[u] = tp, dfn[u] = ++tim, rid[tim] = u;
        if (son[u]) dfs2(son[u], tp);
        for (int v : g[u]) if (v != fa[u] && v != son[u]) dfs2(v, v);
    }
    void build(int rt = 1) { // rt: 树根
        dep[0] = 0, dfs1(rt, 0), dfs2(rt, rt), bit = RBIT<ll>(n);
    }
    void init_val(const vector<ll> &w) { // w[1..n]，仅初始化一次
        for (int u = 1; u <= n; u++) bit.add(dfn[u], dfn[u], w[u]);
    }
    void path_add(int u, int v, ll x) { // 路径 u-v 点权 +=x
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            bit.add(dfn[top[u]], dfn[u], x), u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        bit.add(dfn[u], dfn[v], x);
    }
    ll path_sum(int u, int v) { // 路径 u-v 点权和
        ll z = 0;
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            z += bit.sum(dfn[top[u]], dfn[u]), u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        return z + bit.sum(dfn[u], dfn[v]);
    }
    int lca(int u, int v) const {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
```

```
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
    int jump(int u, int k) const { // u 向上 k 条边；越过根返回 0
        while (u) {
            int d = dep[u] - dep[top[u]];
            if (k <= d) return rid[dfn[u] - k];
            k -= d + 1, u = fa[top[u]];
        }
        return 0;
    }
    int kth_path(int u, int v, int k) const { // u->v 第 k 个点，k=0 是 u
        int w = lca(u, v), a = dep[u] - dep[w], d = a + dep[v] - dep[w];
        return k <= a ? jump(u, k) : jump(v, d - k);
    }
    void sub_add(int u, ll x) { bit.add(dfn[u], dfn[u] + sz[u] - 1, x); }
    ll sub_sum(int u) { return bit.sum(dfn[u], dfn[u] + sz[u] - 1); }
    void root_sub_add(int u, int rt, ll x) { // 换根 rt 后 u 的子树 +=x
        if (u == rt) return bit.add(1, n, x);
        if (lca(u, rt) != u) return sub_add(u, x);
        int v = jump(rt, dep[rt] - dep[u] - 1);
        bit.add(1, n, x), sub_add(v, -x);
    }
    ll root_sub_sum(int u, int rt) { // 换根 rt 后 u 的子树和
        if (u == rt) return bit.sum(1, n);
        if (lca(u, rt) != u) return sub_sum(u);
        int v = jump(rt, dep[rt] - dep[u] - 1);
        return bit.sum(1, n) - sub_sum(v);
    }
};
```

若维护边权，把每条父边权放在较深端点上。查询路径 `u-v` 的最后一段时排除 LCA：使用 `[dfn[u]+1,dfn[v]]`；跨链段仍查询整段，因为链顶对应的父边属于该段。

14.2 DSU on tree：子树颜色统计

又称树上启发式合并或 Sack。先处理轻儿子并清空贡献，最后处理重儿子且保留贡献，再把所有轻子树加入。每个轻边下的节点只会被重新加入 `O(log n)` 次。

下方模板返回每个节点子树内不同颜色数：

- `DsuTree d(n,color,m)`：`color[1..n]` 已离散化到 `[0,m)`。
- `add_edge(u,v)` 建树，`run(root)` 返回 `ans[u]`。
- 要维护“出现次数最多的颜色和”“至少出现 k 次”等，只需改 `add_node` 和记录答案的部分。

预处理和统计 `O(n log n)`，空间 `O(n+m)`。DFS 使用递归，链形树需注意栈深。

```
struct DsuTree {
    int n, tim = 0, now = 0;
    vector<vector<int>> g;
```

```
    vector<int> c, fa, sz, son, st, ed, ord, cnt, ans;
    DsuTree(int n, vector<int> c, int m) : n(n), g(n + 1), c(move(c)),
        fa(n + 1), sz(n + 1), son(n + 1), st(n + 1), ed(n + 1),
        ord(n + 1), cnt(m), ans(n + 1) {}
    void add_edge(int u, int v) { // 无向边 u-v
        g[u].push_back(v), g[v].push_back(u);
    }
    void dfs1(int u, int f) {
        fa[u] = f, sz[u] = 1, st[u] = ++tim, ord[tim] = u;
        for (int v : g[u]) if (v != f) {
            dfs1(v, u), sz[u] += sz[v];
            if (!son[u] || sz[v] > sz[son[u]]) son[u] = v;
        }
        ed[u] = tim;
    }
    void add_node(int u, int d) {
        if (d == 1) now += cnt[c[u]]++ == 0;
        else now -= --cnt[c[u]] == 0;
    }
    void add_sub(int u, int d) {
        for (int i = st[u]; i <= ed[u]; i++) add_node(ord[i], d);
    }
    void dfs(int u, bool keep) {
        for (int v : g[u]) if (v != fa[u] && v != son[u]) dfs(v, false);
        if (son[u]) dfs(son[u], true);
        for (int v : g[u]) if (v != fa[u] && v != son[u]) add_sub(v, 1);
        add_node(u, 1), ans[u] = now;
        if (!keep) add_sub(u, -1);
    }
    vector<int> run(int rt = 1) { // 返回各子树不同颜色数
        dfs1(rt, 0), dfs(rt, true);
        return ans;
    }
};
```

15. Link-Cut Tree：动态森林

LCT 维护森林，支持在线连边、删边、修改点权和查询任意路径异或和。它适用于边动态变化的树；若树静态，树链剖分通常更短、常数更小。

- `LCT t(n)`：建立 `n` 个孤立点。
- `set_val(x,v)`：点 `x` 赋值为 `v`。
- `link(x,y)`：若不连通则加边并返回 `true`。
- `cut(x,y)`：若存在直接边 `x-y` 则删除并返回 `true`。
- `connected(x,y)`：判连通。
- `query(x,y)`：路径异或和，调用前必须连通。

所有操作均摊 `O(log n)`。若要维护路径和，把 `s = s_l xor v xor s_r` 改成加法；若要支持路径加法，还需维护虚实翻转之外的懒标记和子树大小。

```
struct LCT {
    struct N { int ch[2] = {}, f = 0; ll v = 0, s = 0; bool rv = false; };
    vector<N> t;
    LCT(int n = 0) : t(n + 1) {}
    bool root(int x) const {
        int f = t[x].f;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }
    void pull(int x) { t[x].s = t[t[x].ch[0]].s ^ t[x].v ^ t[t[x].ch[1]].s; }
    void rev(int x) {
        if (!x) return;
        swap(t[x].ch[0], t[x].ch[1]), t[x].rv ^= 1;
    }
    void push(int x) {
        if (t[x].rv) rev(t[x].ch[0]), rev(t[x].ch[1]), t[x].rv = false;
    }
    void rotate(int x) {
        int y = t[x].f, z = t[y].f, k = (t[y].ch[1] == x), w = t[x].ch[k ^ 1];
        if (!root(y)) t[z].ch[t[z].ch[1] == y] = x;
        t[x].f = z, t[x].ch[k ^ 1] = y, t[y].f = x;
        t[y].ch[k] = w;
        if (w) t[w].f = y;
        pull(y), pull(x);
    }
    void splay(int x) {
        vector<int> st{x};
        for (int y = x; !root(y); ) y = t[y].f, st.push_back(y);
        while (!st.empty()) push(st.back()), st.pop_back();
        while (!root(x)) {
            int y = t[x].f, z = t[y].f;
            if (!root(y)) ((t[y].ch[1] == x) ^ (t[z].ch[1] == y)) ? rotate(x) :
rotate(y);
            rotate(x);
        }
    }
    void access(int x) {
        for (int y = 0; x; x = t[y = x].f) splay(x), t[x].ch[1] = y, pull(x);
    }
    void makeroot(int x) { access(x), splay(x), rev(x); }
    int findroot(int x) {
        access(x), splay(x), push(x);
        while (t[x].ch[0]) x = t[x].ch[0], push(x);
        splay(x);
        return x;
    }
    bool connected(int x, int y) { return findroot(x) == findroot(y); }
    bool link(int x, int y) { // 加边 x-y
        makeroot(x);
```

```
        if (findroot(y) == x) return false;
        t[x].f = y;
        return true;
    }
    bool cut(int x, int y) { // 删除直接边 x-y
        makeroot(x), access(y), splay(y);
        if (t[y].ch[0] != x || t[x].ch[1]) return false;
        t[y].ch[0] = t[x].f = 0, pull(y);
        return true;
    }
    void set_val(int x, ll v) { // 点 x 赋值 v
        access(x), splay(x), t[x].v = v, pull(x);
    }
    ll query(int x, int y) { // 路径异或和，须连通
        makeroot(x), access(y), splay(y);
        return t[y].s;
    }
};
```

`access` 的紧凑 `for` 写法常见但不易改动：循环末尾先令 `y=x`，再跳到原 `x` 的父亲。需要扩展时可拆成普通变量更新，避免误改赋值顺序。

---

16. KD Tree：二维矩形和与最近点

静态二维点集上，KD Tree 可做矩形统计与欧氏距离最近点。每个节点维护子树包围盒；查询时整盒包含就直接取和，整盒不交就剪枝，最近点则按包围盒距离剪枝。

- 点为 `{x,y,w,id}`，其中 `w` 是矩形求和权值，`id` 用于最近点查询时排除自身。
- `KD kd(points)`：`points` 是普通 `0-based vector<P>`。
- `rect(x1,y1,x2,y2)`：闭矩形点权和。
- `near(x,y,ban)`：到 `(x,y)` 的最小平方距离；`id==ban` 的点不参与，传 `-1` 表示不排除。

建树期望 `O(n log n)`，空间 `O(n)`；随机/一般数据单次查询通常接近 `O(sqrt(n))`，最坏可退化到 `O(n)`。坐标平方和必须使用 `ll`。

```
struct KD {
    struct P { int x, y; ll w; int id; };
    struct N {
        P p{}; int l = 0, r = 0;
        int xl = 0, xr = 0, yl = 0, yr = 0;
        ll s = 0;
    };
    vector<P> a;
    vector<N> t;
    int rt = 0;
    KD(vector<P> p = {}) : a(move(p)), t(a.size() + 1) {
        if (!a.empty()) rt = build(0, (int)a.size() - 1, 0);
    }
    void pull(int u) {
```

```
        auto &q = t[u];
        q.xl = q.xr = q.p.x, q.yl = q.yr = q.p.y, q.s = q.p.w;
        for (int v : {q.l, q.r}) if (v) {
            q.xl = min(q.xl, t[v].xl), q.xr = max(q.xr, t[v].xr);
            q.yl = min(q.yl, t[v].yl), q.yr = max(q.yr, t[v].yr);
            q.s += t[v].s;
        }
    }
    int build(int l, int r, int d) {
        if (l > r) return 0;
        int m = (l + r) >> 1;
        nth_element(a.begin() + l, a.begin() + m, a.begin() + r + 1,
            [d](const P &u, const P &v) { return d ? u.y < v.y : u.x < v.x; });
        int u = m + 1;
        t[u].p = a[m], t[u].l = build(l, m - 1, d ^ 1), t[u].r = build(m + 1, r, d ^
1);
        pull(u);
        return u;
    }
    ll rect(int u, int x1, int y1, int x2, int y2) const {
        if (!u || t[u].xr < x1 || x2 < t[u].xl || t[u].yr < y1 || y2 < t[u].yl)
return 0;
        if (x1 <= t[u].xl && t[u].xr <= x2 && y1 <= t[u].yl && t[u].yr <= y2) return
t[u].s;
        ll z = rect(t[u].l, x1, y1, x2, y2) + rect(t[u].r, x1, y1, x2, y2);
        auto p = t[u].p;
        if (x1 <= p.x && p.x <= x2 && y1 <= p.y && p.y <= y2) z += p.w;
        return z;
    }
    ll rect(int x1, int y1, int x2, int y2) const { // 闭矩形权值和
        return rect(rt, x1, y1, x2, y2);
    }
    ll box(int u, int x, int y) const {
        if (!u) return INF;
        ll dx = max({(ll)t[u].xl - x, 0LL, (ll)x - t[u].xr});
        ll dy = max({(ll)t[u].yl - y, 0LL, (ll)y - t[u].yr});
        return dx * dx + dy * dy;
    }
    void near(int u, int x, int y, int ban, ll &ans) const {
        if (!u || box(u, x, y) >= ans) return;
        auto p = t[u].p;
        if (p.id != ban) {
            ll dx = (ll)p.x - x, dy = (ll)p.y - y;
            ans = min(ans, dx * dx + dy * dy);
        }
        int l = t[u].l, r = t[u].r;
        if (box(l, x, y) > box(r, x, y)) swap(l, r);
        near(l, x, y, ban, ans), near(r, x, y, ban, ans);
    }
    ll near(int x, int y, int ban = -1) const { // 最小平方距离
        ll ans = INF;
        near(rt, x, y, ban, ans);
```

```
        return ans;
    }
};
```

若坐标差可能超过约 `2e9`，平方可能溢出 `ll`，应在距离计算处改用 `__int128`。大量在线插入时需加入替罪羊式重构；静态题不要为此增加代码。

---

17. 李超树：动态直线/线段最值

维护若干一次函数 `y=kx+b`，回答指定横坐标处的最小值或最大值。下方是离散坐标版本：先收集所有可能查询的 `x`，可比在巨大整数值域上动态开点更省内存、常数更小。

- `LiChao<true> tr(xs)`：维护最小值；`LiChao<false>` 维护最大值。`xs` 可无序、有重复，构造时会排序去重。
- `add_line(k,b,id)`：加入在全部查询坐标上生效的直线；`id` 可省略，默认使用内部编号。
- `add_seg(k,b,xl,xr,id)`：只在闭区间 `[xl,xr]` 内生效；实际覆盖其中存在于 `xs` 的坐标。
- `ask(x)`：返回 `{最优函数值,直线 id}`；`x` 必须在构造时的 `xs` 中。
- `i128` 比较避免 `k*x+b` 中间溢出，答案也保留为 `i128`；`str128` 用于输出。

整条直线插入 `O(log n)`，线段插入 `O(log^2 n)`，单点查询 `O(log n)`；空间 `O(n+q)`。相同最优值时返回任意一条线。

```
using i128 = __int128_t;

string str128(i128 x) { // 输出 i128
    if (!x) return "0";
    bool neg = x < 0;
    if (neg) x = -x;
    string s;
    while (x) s.push_back('0' + x % 10), x /= 10;
    if (neg) s.push_back('-');
    reverse(s.begin(), s.end());
    return s;
}

template<bool MIN = true> struct LiChao {
    struct L {
        ll k, b;
        int id;
        i128 get(ll x) const { return (i128)k * x + b; }
    };
    int n;
    vector<ll> x;
    vector<L> ln;
    vector<int> t;
    LiChao(vector<ll> xs = {}) : x(move(xs)) {
        sort(x.begin(), x.end());
        x.erase(unique(x.begin(), x.end()), x.end());
```

```
        n = x.size(), t.assign(4 * max(1, n), 0);
        ln.push_back({0, 0, -1});
    }
    i128 inf() const { return (i128)1 << 126; }
    bool better(i128 a, i128 b) const { return MIN ? a < b : a > b; }
    bool better(int a, int b, ll px) const { // line a 是否优于 b
        if (!a) return false;
        if (!b) return true;
        return better(ln[a].get(px), ln[b].get(px));
    }
    int line(ll k, ll b, int id) {
        int p = ln.size();
        ln.push_back({k, b, id < 0 ? p : id});
        return p;
    }
    void ins(int q, int o, int l, int r) { // 节点区间插入整条线 q
        int &p = t[o];
        if (!p) return void(p = q);
        int m = (l + r) >> 1;
        if (better(q, p, x[m])) swap(q, p);
        if (l == r) return;
        if (better(q, p, x[l])) ins(q, o << 1, l, m);
        else if (better(q, p, x[r])) ins(q, o << 1 | 1, m + 1, r);
    }
    int add_line(ll k, ll b, int id = -1) { // 全域加入 y=kx+b
        int q = line(k, b, id);
        if (n) ins(q, 1, 0, n - 1);
        return q;
    }
    void seg(int q, int ql, int qr, int o, int l, int r) {
        if (ql <= l && r <= qr) return ins(q, o, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) seg(q, ql, qr, o << 1, l, m);
        if (m < qr) seg(q, ql, qr, o << 1 | 1, m + 1, r);
    }
    int add_seg(ll k, ll b, ll xl, ll xr, int id = -1) { // [xl,xr] 生效
        int l = lower_bound(x.begin(), x.end(), xl) - x.begin();
        int r = upper_bound(x.begin(), x.end(), xr) - x.begin() - 1;
        if (l > r) return 0;
        int q = line(k, b, id);
        seg(q, l, r, 1, 0, n - 1);
        return q;
    }
    pair<i128, int> ask(int p, int o, int l, int r) const {
        pair<i128, int> z = {MIN ? inf() : -inf(), -1};
        if (t[o]) z = {ln[t[o]].get(x[p]), ln[t[o]].id};
        if (l == r) return z;
        int m = (l + r) >> 1;
        auto w = p <= m ? ask(p, o << 1, l, m) : ask(p, o << 1 | 1, m + 1, r);
        return better(w.first, z.first) ? w : z;
    }
    pair<i128, int> ask(ll px) const { // px 必须在 xs 中
```

```
        int p = lower_bound(x.begin(), x.end(), px) - x.begin();
        assert(p < n && x[p] == px);
        return ask(p, 1, 0, n - 1);
    }
};
```

最小值示例：`LiChao<true> tr(xs); tr.add_line(k,b,id); auto [y,who]=tr.ask(x);`。若 `who==-1`，说明该坐标没有任何生效直线。题目保证答案在 `ll` 范围内时可强转输出，否则使用 `cout << str128(y)`。

若斜率和横坐标本身可能超过 `ll`，应把 `L::k` 和坐标类型也改为 `i128`；仅把计算结果改成 `i128` 不能挽救输入类型本身的溢出。

---

18. 珂朵莉树 ODT

ODT 用有序集合维护若干值相同的连续区间，区间赋值时删除被覆盖区间，因此只有在数据随机、区间赋值足够多、区间数长期较少时才快。它没有可靠的多项式最坏复杂度保证，构造数据可使其退化到每次 `O(n)`；严肃的强制在线题不要仅凭“有区间赋值”就使用。

- `ODT odt(n,v)`：建立 `a[1..n]=v`。
- `assign(l,r,v)`：区间赋值。
- `add(l,r,v)`：区间加。
- `kth(l,r,k)`：按重复次数计算的第 `k` 小，`k` 从 `1` 开始。
- `power_sum(l,r,e,mod)`：`sum(a[i]^e) mod mod`。

`split(p)` 把包含 `p` 的区间切开并返回左端点为 `p` 的迭代器，是所有操作的核心。复杂度取决于遍历和删除的区间数。

```
struct ODT {
    struct N {
        int l, r;
        mutable ll v;
        bool operator<(const N &o) const { return l < o.l; }
    };
    int n;
    set<N> s;
    ODT(int n, ll v = 0) : n(n) { s.insert({1, n, v}); }
    auto split(int p) { // 切出左端点 p；p=n+1 返回 end
        if (p > n) return s.end();
        auto it = s.lower_bound({p, 0, 0});
        if (it != s.end() && it->l == p) return it;
        --it;
        int l = it->l, r = it->r; ll v = it->v;
        s.erase(it), s.insert({l, p - 1, v});
        return s.insert({p, r, v}).first;
    }
    void assign(int l, int r, ll v) { // a[l..r]=v
        auto y = split(r + 1), x = split(l);
```

```
        s.erase(x, y), s.insert({l, r, v});
    }
    void add(int l, int r, ll v) { // a[l..r]+=v
        auto y = split(r + 1), x = split(l);
        for (auto it = x; it != y; ++it) it->v += v;
    }
    ll kth(int l, int r, int k) { // [l,r] 第 k 小，1-based
        auto y = split(r + 1), x = split(l);
        vector<pair<ll, int>> a;
        for (auto it = x; it != y; ++it) a.push_back({it->v, it->r - it->l + 1});
        sort(a.begin(), a.end());
        for (auto [v, c] : a) if ((k -= c) <= 0) return v;
        return INF;
    }
    ll qpow(ll a, ll b, ll mod) const {
        ll z = 1 % mod;
        for (a = (a % mod + mod) % mod; b; b >>= 1, a = (ll)((__int128)a * a % mod))
            if (b & 1) z = (ll)((__int128)z * a % mod);
        return z;
    }
    ll power_sum(int l, int r, ll e, ll mod) { // sum(a[i]^e) mod mod
        auto y = split(r + 1), x = split(l);
        ll z = 0;
        for (auto it = x; it != y; ++it)
            z = (z + (__int128)qpow(it->v, e, mod) * (it->r - it->l + 1)) % mod;
        return z;
    }
};
```

区间加不会减少区间数，区间赋值才会大量合并式删除旧区间。上方短版不主动合并相邻同值区间；若操作会频繁制造相同邻段，可在 `assign` 后检查前后邻居并手动合并，以改善常数。

---

19. 离线区间与偏序算法

19.1 普通莫队：静态区间不同数个数

莫队把离线询问重排，使当前区间的左右端点移动次数较少。要求答案能在加入/删除一个元素时快速维护。

下方函数解决每个闭区间 `[l,r]` 有多少种不同值：`a[1..n]` 必须已离散化到 `[0,m)`，`qs[id]={l,r}`，返回值按原询问顺序排列。使用奇偶块排序降低常数。

时间约 `O((n+q)sqrt(n))`，空间 `O(n+m+q)`；端点每移动一步是 `O(1)`。若加入/删除代价不对称，应调整块长。

```
vector<int> mo_distinct(const vector<int> &a, int m,
                        const vector<pair<int, int>> &qs) { // a[1..n], query [l,r]
    int n = (int)a.size() - 1, qn = qs.size();
    if (!qn) return {};
    int B = max(1, (int)(n / sqrt(qn)));
```

```
    struct Q { int l, r, id; };
    vector<Q> q(qn);
    for (int i = 0; i < qn; i++) q[i] = {qs[i].first, qs[i].second, i};
    sort(q.begin(), q.end(), [&](const Q &x, const Q &y) {
        int a = x.l / B, b = y.l / B;
        if (a != b) return a < b;
        return a & 1 ? x.r > y.r : x.r < y.r;
    });
    vector<int> cnt(m), ans(qn);
    int l = 1, r = 0, now = 0;
    auto add = [&](int i) { if (!cnt[a[i]]++) now++; };
    auto del = [&](int i) { if (!--cnt[a[i]]) now--; };
    for (auto [ql, qr, id] : q) {
        while (l > ql) add(--l);
        while (r < qr) add(++r);
        while (l < ql) del(l++);
        while (r > qr) del(r--);
        ans[id] = now;
    }
    return ans;
}
```

要维护区间出现次数平方和、相同数对数等，只需改 `add/del` 对答案的增量。通常应先删除旧贡献、改频次、再加入新贡献。

19.2 带修改莫队：区间不同数个数

给每个询问增加时间维 `t`，表示它之前发生了多少次单点修改。排序后同时移动左端点、右端点和时间。

- `MC{p,old,now}`：位置 `p` 从 `old` 改成 `now`；`c[0]` 放一个占位元素。
- `MQ{l,r,t,id}`：询问闭区间，`t` 是其之前的修改数。
- 初始数组和所有新旧值都应离散化到 `[0,m)`。

读入时用一份临时数组记录当前值：遇到修改 `(p,x)`，保存 `{p,tmp[p],x}` 后令 `tmp[p]=x`；遇到询问则保存当前 `c.size()-1` 为 `t`。

常用块长为 `n^(2/3)`，总复杂度约 `O((n+q)n^(2/3))`。

```
struct MQ { int l, r, t, id; };
struct MC { int p, old, now; };

vector<int> mo_change_distinct(vector<int> a, int m, vector<MQ> q,
                               const vector<MC> &c) { // c[0] 占位
    int n = (int)a.size() - 1;
    int B = max(1, (int)pow(max(1, n), 2.0 / 3));
    sort(q.begin(), q.end(), [&](const MQ &x, const MQ &y) {
        int a = x.l / B, b = y.l / B;
        if (a != b) return a < b;
        int c = x.r / B, d = y.r / B;
        if (c != d) return (a & 1) ? c > d : c < d;
```

```
        return (c & 1) ? x.t > y.t : x.t < y.t;
    });
    int qn = 0;
    for (auto x : q) qn = max(qn, x.id + 1);
    vector<int> cnt(m), ans(qn);
    int l = 1, r = 0, tim = 0, cur = 0;
    auto addv = [&](int x) { if (!cnt[x]++) cur++; };
    auto delv = [&](int x) { if (!--cnt[x]) cur--; };
    auto add = [&](int i) { addv(a[i]); };
    auto del = [&](int i) { delv(a[i]); };
    auto change = [&](int id, bool go) { // go=1 前进，go=0 撤销
        int p = c[id].p, v = go ? c[id].now : c[id].old;
        if (l <= p && p <= r) delv(a[p]), addv(v);
        a[p] = v;
    };
    for (auto [ql, qr, qt, id] : q) {
        while (l > ql) add(--l);
        while (r < qr) add(++r);
        while (l < ql) del(l++);
        while (r > qr) del(r--);
        while (tim < qt) change(++tim, true);
        while (tim > qt) change(tim--, false);
        ans[id] = cur;
    }
    return ans;
}
```

这里先移动空间再移动时间也正确，因为 `change` 会检查修改点是否在当前区间内。答案若依赖位置而不仅是值，`add/del` 必须接收位置并相应重写。

19.3 CDQ 分治：三维偏序

对每个点 `(x,y,z)`，统计满足 `x'<=x, y'<=y, z'<=z` 的点数。常用于三维偏序、带时间维的二维点修改/矩形询问。

- `dominance3(p)`：`p[i]={x,y,z}`，返回每个点的支配计数，包含点自身。
- 完全相同的点会先合并权重，再把同一答案还原给所有副本。
- 第一维排序，第二维 CDQ 归并，第三维离散化后用树状数组统计。

复杂度 `O(n log^2 n)`，空间 `O(n)`。若题目要求严格小于，要根据哪一维严格调整排序、分组和归并时的 `<=`。

```
vector<int> dominance3(const vector<array<int, 3>> &p) { // count x'<=x,y'<=y,z'<=z
    struct N { int x, y, z, w, ans, g; };
    int n = p.size();
    vector<array<int, 4>> q(n);
    for (int i = 0; i < n; i++) q[i] = {p[i][0], p[i][1], p[i][2], i};
    sort(q.begin(), q.end());
    vector<N> a;
    vector<int> gid(n), zs;
    for (auto [x, y, z, id] : q) {
```

```
        if (a.empty() || a.back().x != x || a.back().y != y || a.back().z != z) {
            int g = a.size();
            a.push_back({x, y, z, 1, 0, g}), zs.push_back(z), gid[id] = g;
        } else a.back().w++, gid[id] = a.back().g;
    }
    sort(zs.begin(), zs.end());
    zs.erase(unique(zs.begin(), zs.end()), zs.end());
    for (auto &u : a) u.z = lower_bound(zs.begin(), zs.end(), u.z) - zs.begin() + 1;
    vector<N> tmp(a.size());
    BIT<int> bit(zs.size());
    auto cdq = [&](auto self, int l, int r) -> void {
        if (l >= r) return;
        int m = (l + r) >> 1;
        self(self, l, m), self(self, m + 1, r);
        int i = l;
        for (int j = m + 1; j <= r; j++) {
            while (i <= m && a[i].y <= a[j].y) bit.add(a[i].z, a[i].w), i++;
            a[j].ans += bit.sum(a[j].z);
        }
        for (int k = l; k < i; k++) bit.add(a[k].z, -a[k].w);
        i = l;
        int j = m + 1, k = l;
        while (i <= m && j <= r) tmp[k++] = a[i].y <= a[j].y ? a[i++] : a[j++];
        while (i <= m) tmp[k++] = a[i++];
        while (j <= r) tmp[k++] = a[j++];
        for (k = l; k <= r; k++) a[k] = tmp[k];
    };
    if (!a.empty()) cdq(cdq, 0, (int)a.size() - 1);
    vector<int> by_group(a.size()), ans(n);
    for (auto u : a) by_group[u.g] = u.ans + u.w;
    for (int i = 0; i < n; i++) ans[i] = by_group[gid[i]];
    return ans;
}
```

19.4 树上莫队：路径不同颜色数

将树做长度 `2n` 的括号式 Euler 序，每个点进入和离开各出现一次。移动区间端点时切换该点是否有效，就能把路径询问转成普通莫队。

`mo_tree_distinct(g,a,m,qs,rt)` 中 `g` 是 `1..n` 的树，`a[u]` 是离散化到 `[0,m)` 的颜色，`qs[i]={u,v}` 是路径端点。函数返回每条闭路径上不同颜色数，内部已完成迭代 DFS、倍增 LCA 和额外 LCA 的临时加入。

复杂度约 $O((n+q)\sqrt{n}+n\log n+q\log n)$，空间 $O(n\log n+m+q)$。若询问可在线，或带修改且数据很大，应换树链结构或带修改树上莫队；本段只处理静态树和静态点权。

```
vector<int> mo_tree_distinct(const vector<vector<int>> &g,
                             const vector<int> &a, int m,
                             const vector<pair<int, int>> &qs, int rt = 1) {
    // g：1-index 树；a：颜色；m：颜色数；qs：路径端点；rt：根
```

```cpp
int n = (int)g.size() - 1, lg = 1;
while ((1 << lg) <= n) lg++;
vector<vector<int>> up(lg, vector<int>(n + 1));
vector<int> dep(n + 1), tin(n + 1), tout(n + 1), e(2 * n + 1);
vector<tuple<int, int, int>> st{{rt, 0, 0}};
int tim = 0;
while (!st.empty()) {
    auto [u, f, out] = st.back(); st.pop_back();
    if (!out) {
        tin[u] = ++tim, e[tim] = u, up[0][u] = f;
        dep[u] = dep[f] + 1;
        for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
        st.push_back({u, f, 1});
        for (int i = (int)g[u].size() - 1; i >= 0; i--)
            if (g[u][i] != f) st.push_back({g[u][i], u, 0});
    } else tout[u] = ++tim, e[tim] = u;
}

auto lca = [&](int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int j = 0; j < lg; j++) if (d >> j & 1) u = up[j][u];
    if (u == v) return u;
    for (int j = lg - 1; j >= 0; j--)
        if (up[j][u] != up[j][v]) u = up[j][u], v = up[j][v];
    return up[0][u];
};

struct Q { int l, r, id, ex; };
vector<Q> q;
for (int i = 0; i < (int)qs.size(); i++) {
    auto [u, v] = qs[i];
    if (tin[u] > tin[v]) swap(u, v);
    int p = lca(u, v);
    if (p == u) q.push_back({tin[u], tin[v], i, 0});
    else q.push_back({tout[u], tin[v], i, p});
}
int B = max(1, (int)sqrt(2 * n));
sort(q.begin(), q.end(), [&](const Q &x, const Q &y) {
    int a = x.l / B, b = y.l / B;
    if (a != b) return a < b;
    return a & 1 ? x.r > y.r : x.r < y.r;
});

vector<int> cnt(m), ans(qs.size());
vector<char> on(n + 1);
int l = 1, r = 0, now = 0;
auto flip = [&](int p) {
    int u = e[p], c = a[u];
    if (on[u]) { if (!--cnt[c]) now--; }
    else { if (!cnt[c]++) now++; }
    on[u] ^= 1;
```

```
    };
    for (auto [ql, qr, id, ex] : q) {
        while (l > ql) flip(--l);
        while (r < qr) flip(++r);
        while (l < ql) flip(l++);
        while (r > qr) flip(r--);
        if (ex) {
            int c = a[ex];
            if (!cnt[c]++) now++;
            ans[id] = now;
            if (!--cnt[c]) now--;
        } else ans[id] = now;
    }
    return ans;
}
```

20. 可删除堆

两个普通堆做懒删除：`a` 保存所有插入，`b` 保存待删除元素；两边堆顶相同就一起弹出。适用于只需堆顶、且保证删除的元素当前存在的场景。

- `push(x)`、`erase(x)`：插入与标记删除。
- `top()`、`pop()`：读取/删除当前最小值。
- 模板默认小根堆；若要大根堆，把 `greater<T>` 改为 `less<T>`。

每次操作均摊 `O(log n)`。懒删除元素在到达堆顶前仍占内存，因此总空间按操作数计算。

```
template<class T> struct DelHeap {
    priority_queue<T, vector<T>, greater<T>> a, b;
    int sz = 0;
    void clean() {
        while (!a.empty() && !b.empty() && a.top() == b.top()) a.pop(), b.pop();
    }
    void push(T x) { a.push(x), sz++; } // 插入 x
    void erase(T x) { b.push(x), sz--; } // 删除一个存在的 x
    T top() { clean(); return a.top(); } // 非空时调用
    void pop() { clean(); a.pop(), sz--; } // 删除堆顶
    int size() const { return sz; }
    bool empty() const { return sz == 0; }
};
```

若删除一个不存在的值，两个堆会永久错配；需要任意可靠删除时使用 `multiset` 或有序 Treap。

21. 高性能哈希表

`unordered_map` 在特殊数据下可能因大量碰撞退化。SplitMix64 给整数键加随机扰动，可显著降低被构造碰撞的风险。

```cpp
struct Hash {
    static uint64_t mix(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t sd =
chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + sd);
    }
    template<class A, class B> size_t operator()(const pair<A, B> &p) const {
        return mix((uint64_t)p.first + 0x9e3779b97f4a7c15ULL * (uint64_t)p.second);
    }
};

unordered_map<ll, int, Hash> mp;
unordered_set<ll, Hash> st;
```

已知元素数量时先调用 `mp.reserve(n*2)` 并设置 `mp.max_load_factor(0.7)`，能减少扩容和冲突。键可离散化时，数组通常比哈希表更快、更省内存、更稳定。

---

22. 常见技巧与结论

22.1 `bitset` 加速

固定上限布尔集合可用机器字并行：交、并、异或与计数约为 `O(N/64)`。

```cpp
const int N = 200005;
bitset<N> a, b;
int inter = (a & b).count(); // 交集大小
bool hit = (a & b).any();    // 是否有交集
```

01 背包可达性：`f[i]=1` 表示和 `i` 可达，每个物品权值 `w` 执行 `f |= f << w`。复杂度约 `O(nV/64)`。

```cpp
const int V = 1000000;
bitset<V + 1> f;
f[0] = 1;
for (int w : weight) f |= f << w; // 每个 w 只能选一次
```

动态图可达闭包在 `n` 较小时，也可让每个点维护一个 `bitset`，按拓扑序反向执行并集。

22.2 环形序列偏移

无需真的移动元素。对 `0-based` 长度 `n` 的数组维护偏移 `sh`，逻辑下标 `i` 映射为物理下标 `(i+sh)%n`。区间映射后若左端点大于右端点，就拆成尾段与头段两个区间。

```
sh = (sh + x % n + n) % n; // 偏移 x，可为负
int L = (l + sh) % n, R = (r + sh) % n; // 逻辑 [l,r]
if (L <= R) work(L, R);
else work(L, n - 1), work(0, R);
```

必须先确认题意中的"左移/右移"与映射方向；不同定义可能要用 `sh-x`。

22.3 删除操作转反向加入

并查集等结构擅长加入、不擅长删除。如果全部操作离线且只有删除，可以先执行完所有删除得到最终状态，再倒序把被删元素或边加回来，答案最后反转。

- 同一元素可能删除多次时要记录有效删除次数，只有计数从 `1` 变为 `0` 才真正恢复。
- 同时有任意插入和删除时，可把每条边的生存时间拆成若干区间，配合可撤销并查集做时间轴分治。
- 若答案依赖操作顺序且无法离线还原，就不能使用反向技巧。

22.4 小并大 / 启发式合并

合并两个集合时，总把较小集合的元素逐个插入较大集合。每个元素所在集合的大小每移动一次至少翻倍，因此每个元素最多移动 `O(log n)` 次。

```
if (a.size() < b.size()) swap(a, b); // a 保持为大集合
for (auto x : b) a.insert(x);
b.clear();
```

若容器单次插入是 `O(1)` 均摊，总复杂度常为 `O(n log n)`；使用 `set/map` 时还要乘单次插入的 `log n`。树上合并颜色集合、合并 Trie、维护子树信息都常用此结论。

22.5 整体二分与并行二分

当大量询问的答案位于同一有序值域，并且"答案是否 `<=mid`"可以通过扫描事件和树状数组等结构批量判断时，可把所有询问一起二分。

- 整体二分按 `mid` 把修改和询问稳定分到左右两侧，递归前撤销本层修改；常见复杂度 `O((n+q)log V log n)`。
- 并行二分为每个询问维护 `[lo,hi]`，每轮按中点分桶并重新扫描事件；代码通常更直观，但会多次清空结构。
- 判定必须对答案单调；修改的时间顺序、严格/非严格边界必须在事件排序中保持。

典型问题包括静态/动态第 k 小、最早满足条件的时刻、区间内第 k 个事件。若仅静态区间第 k 小，小波矩阵通常更短更快。

22.6 相等元素的贡献归属

用单调栈统计“每个元素作为区间最小/最大值的贡献”时，两侧不能同时使用同一种相等规则，否则会重复或遗漏。常用规则是：左侧找严格更小，右侧找小于等于；或反过来。只要一侧严格、一侧非严格即可让每个区间唯一归属。

22.7 未收入主模板的结构

- Splay 的普通有序集和序列功能已分别由优化 FHQ、隐式 FHQ 覆盖；LCT 内部已经包含必要的 Splay。
- 二维 ST 表查询快但空间常达 `O(nm log n log m)`，实际更常用二维前缀、离线扫描、KD Tree 或按一维建 ST。
- 动态 KD Tree、Top Tree、Euler Tour Tree、van Emde Boas 等结构使用频率低、实现风险高，遇到明确题型再单独准备更合适。
- 普通和可持久化线段树按本板子约定不收入；线段树只保留无法被现有结构等价替代的李超树。
- Aho-Corasick、后缀数组/自动机属于字符串板子；高精度、取模类和分数类属于数论或基础工具板子。

---

23. 赛场检查清单

1. 下标是 `0-based` 还是 `1-based`，区间是闭区间还是半开区间。
2. 所有区间和、计数乘长度、距离平方是否使用 `ll`；乘法取模是否需要 `__int128`。
3. 树状数组下标不能为 `0`；`kth(k)` 要求频次非负且 `k` 合法。
4. 可撤销并查集不能路径压缩；带权并查集先明确权值方向。
5. Treap 预留节点数按总插入次数计算，删除未回收时尤其如此。
6. PBDS 多重集使用 `(value,id)`，不要使用 `less_equal`。
7. ST 表只用于幂等运算；一般结合律运算用 Disjoint Sparse Table。
8. 小波矩阵与莫队的值先离散化，确认第 k 小的 `k` 从 `0` 还是 `1` 开始。
9. Trie 删除前保证元素存在，LCT 的 `cut(x,y)` 只删除直接边。
10. ODT 和 KD Tree 都可能退化；确认数据性质允许使用。
11. 递归 DFS、Treap、KD Tree 在极端结构下是否会爆栈。
12. 空集合的前驱、后继、堆顶、最近点是否需要特判。
13. 李超树的所有查询横坐标是否已加入 `xs`，线段生效范围是否为闭区间。
14. 持久化结构是否只修改了克隆节点，旧版本根是否仍保持不变。
15. 区间赋值与区间加法同时存在时，赋值是否清空旧加法标记。
16. CDQ、整体二分中的相等坐标究竟使用 `<` 还是 `<=`。
17. 来源追踪线性基的 `id` 是否唯一且小于 `MAXN`，位集内存是否符合限制。